



UNIVERSITY OF
LIVERPOOL

Comp 390 Honours Project

2025/2026

Building an AI-Powered Song Writing Tool: Enhancing Creativity Without Impeding Humanity

By

Louis J Selwood

201738332



BSc Computer Science

School of Computer Science & Informatics

University of Liverpool

L69 3DR

Statement of ethical compliance

Dedicated to Emily

Acknowledgements

I would like to thank my parents for their unwavering support during my studies.

Part of this project made use of the pre-trained models MusicLang and Vicuna. Permission to use these models was granted under the General Purpose License and the Non-Commercial License respectively.



Comp 390 Honours Project

2025/2026

Building an AI-Powered Song Writing Tool:

Enhancing Creativity Without Impeding Humanity

BSc Computer Science

School of Computer Science & Informatics

University of Liverpool

L69 3DR

Statement of ethical compliance

Data Category A: Anonymous data will be used from an open source dataset.

Participant Category 2: Humans used during software evaluation

I confirm that this project will follow the ethical guidelines given by the University of Liverpool

Abstract

AI-driven music tools have long been a subject of debate in the music industry due to their powerful generative capabilities and the potential they hold to replace or diminish the role of human creativity in the composition process. This project aims to address this problem by integrating a proposed dual-model AI system, consisting of a generative and explanatory model into a streamlined, interactive user interface designed to support composition workflow while preserving the composer's creative agency.

The resulting software, named CeCe, implements a custom temperature varying chord suggestion algorithm on top the MusicLang generative model, a four tier hierarchical song model which gives the user high levels of compositional control, and a streamlined user interface designed to present AI suggestions as non-intrusively as possible. Evaluation of the generative model demonstrated full chromatic and qualitative distribution which aligns with western music across generated suggestions, with an estimated 2.88×10^{19} unique twelve-chord sequences producible by the sequence. However the explanatory model, based on Vicuna, was found unsuitable for integration due to large inference times of 55-58 seconds and consistently unreliable harmonic analysis, leaving the secondary explanatory component of the dual-model system incomplete

A small-scale usability study of four participants returned positive results across all areas of the software, with AI integration scoring a mean of 4.5 out of 5, suggesting the system successfully avoids the over-automation characteristic of existing AI music tools. The project concludes that CeCe partially achieves its goal of preserving creative agency, with the suggestion algorithm and hierarchical song model forming a credible foundation for a non-intrusive AI composition tool. The dual-model concept remains sound, and future development incorporating a server-side explanatory model represents the most promising direction for delivering on the full original vision.

Table of Contents

1. Introduction & Background	10
2. Requirements Analysis	12
3. Design & Implementation	13
3a. System Architecture Design	14
3b. Implementation of the Global Store	15
3c. The Window System	16
3d. Designing and Implementing the View Layer	18
3e. Development of the Primary Model	18
3f. Implementing the Hierarchical Song Model	19
3g. MIDI System Implementation	21
3h. Implementing the Primary Model	22
3i. Designing & Implementing the Chord Workshop	23
3j. The Secondary Model	26
3k. File Management System	27
3l. Refining the User Interface	27
4. Evaluation & Testing	29
4a. Functional Requirement Evaluation	29
4b. Evaluation & Testing of the Primary Model	34
4c. Evaluation & Testing of the Secondary Model	38
4d. Evaluation & User Testing of the User Interface	41
4e. Evaluation of Creative Agency Preservation	46
5. Project Ethics	47
6. Conclusion & Future Work	48
6a. Summary of Achievements	48
6b. Limitations	50
6c. Future Work	50
6d. Final Conclusion	51
7. BCS Criteria and Critical Reflection	52
References	55
Appendix	56

Technical Terms Glossary

Choice Proliferation: expanding the number of creative decisions available to the user in a way that enables unique, personalised compositions.

Alternating Co-creativity: A mode of human-computer co-creativity in which the human and system take turns contributing to the creative output, each building on what the other has produced.

Autoregressive Model: A type of machine learning model that generates output one token at a time, with each new token predicted based on all previously generated tokens.

Bass Note: In a chord, the lowest-sounding note. When different from the root note, the chord is said to be in an inversion.

Borrowed Chord: A chord taken from the parallel major or minor key, used temporarily within a piece to add harmonic colour or variety. Also referred to as a chromatic chord or modal mixture.

Cadence: A harmonic sequence of chords that creates a sense of resolution or pause in a musical phrase. Common types include perfect (V-I) and deceptive (V-vi) cadences.

Chord Quality: The characteristic sound of a chord, determined by the intervals between its notes. Common qualities include major, minor, diminished, augmented, and dominant seventh.

Chord Sequence: An ordered series of chords used as the harmonic foundation of a musical section or piece. Also referred to as a chord progression.

Chordinomicon: A publicly available open-source dataset containing over 666,000 unique chord progressions spanning multiple musical genres, used in this project during model research.

Chromatic: Relating to notes or chords outside the diatonic scale of a given key. Chromatic elements introduce tones not native to the key signature.

Creative Agency: The degree to which a user retains meaningful control over creative decisions within a system. High creative agency means the user directs the output; low agency means the system determines it.

Diatonic: Relating to notes, chords, or scales that belong naturally to a given key signature, without the use of accidentals or borrowed tones.

Dual-model System: The architectural concept at the centre of CeCe, comprising a primary generative model for producing chord suggestions and a secondary explanatory model for providing music theory context for those suggestions.

Electron: An open-source framework for building cross-platform desktop applications using web technologies (HTML, CSS, JavaScript). CeCe's frontend is built using Electron with React.

Finite State Machine (FSM): A computational model in which a system can exist in one of a finite number of states, transitioning between them based on defined inputs or events. Used in CeCe to govern window behaviour.

Hallucination (AI): A phenomenon in which a language model generates plausible-sounding but factually incorrect or fabricated output. A key failure mode of the secondary model used in this project.

Harmonic Function: The role a chord plays within a key, typically categorised as tonic (stability), subdominant (tension building), or dominant (tension requiring resolution).

Human-Computer Co-creativity: A field of research concerned with systems in which humans and computers collaborate iteratively to produce creative outputs, with neither party fully determining the result.

Key Signature: A set of sharps or flats placed at the beginning of a piece of music, indicating which notes are to be consistently raised or lowered throughout, and defining the tonal centre of the piece.

Log Probability: The natural logarithm of a probability value. Used in this project to score and rank chord suggestions produced by the primary model, as log probabilities are more numerically stable than raw probabilities.

Logits: The raw, unnormalized output values produced by a neural network before a SoftMax function converts them into a probability distribution. Used directly in the temperature-varying sampling algorithm.

MIDI (Musical Instrument Digital Interface): A technical standard that allows electronic instruments and software to communicate. MIDI data encodes musical information such as note pitch, duration, and velocity rather than audio waveforms.

MusicLang: An open-source GPT-2 based chord prediction model used as CeCe's primary generative model. It operates on tokenised chord sequences and predicts musically plausible continuations.

Publish/Subscribe (Pub/Sub): A messaging pattern in which components (subscribers) register interest in events, and are automatically notified by a publisher when those events occur. Used in CeCe's state management layer to propagate updates to the UI.

Quantised Model: A compressed version of a neural network in which weights are stored at reduced numerical precision, reducing memory usage and inference time at the cost of some accuracy.

Roman Numerals (Music Theory): A notation system used in harmonic analysis to label chords by their scale degree relative to the key. Uppercase numerals (I, IV, V) denote major chords; lowercase (ii, iii, vi) denote minor chords.

Root Note: The fundamental note after which a chord is named, and from which its other notes are measured by interval. For example, in a C major chord, C is the root note.

Song Space: A CeCe-specific data structure that provides a chronologically ordered representation of the composition, mapping each beat position to its tempo, time signature, key, and chord content.

Temperature (Sampling): A parameter that controls the randomness of a model's output during token sampling. Lower temperatures produce more predictable, high-probability outputs; higher temperatures introduce greater variety and less conventional results.

Tempo: The speed of a musical piece, measured in beats per minute (BPM). In CeCe, tempo can be set independently for each sequence within a composition.

Time Signature: A notational convention that defines the number of beats in each bar and the note value that counts as one beat (e.g. 4/4, 3/4, 6/8).

Tokenisation: The process of converting input data into discrete tokens that a machine learning model can process. In CeCe, chords are tokenised into six component tokens before being passed to MusicLang.

Vicuna: An open-source instruction-following language model based on LLaMA, used in this project as the secondary explanatory model. Found to be unsuitable for integration due to slow inference times and inconsistent harmonic analysis.

Voice Leading: The way individual notes in successive chords move from one to the next. Smooth voice leading minimises large melodic jumps between chord tones and is a core principle of tonal harmony.

z-order: The visual stacking order of overlapping elements in a graphical interface. Elements with a higher z-order appear in front of those with a lower z-order. Used in CeCe to manage which window appears on top.

1. Introduction & Background

The rapid integration of AI into both technical and creative fields has generated extensive debate regarding the role such systems should play in society. Researchers show that advances in large language models and other generative systems have positioned AI as a general-purpose technology likely to become a standard component of workplace tools, comparable to the adoption of computing or electricity [2]. Similar trends are evident in creative industries, where studies report that AI is increasingly regarded as a co-creative partner across media such as film, literature, music, and visual art, rather than a standalone generator [1]. However, the power of these systems to aid and enhance creative workflows often comes at the cost of reduced humanity in the resulting output. Several studies argue that AI-generated creative work lacks the intentionality, emotional grounding, and experiential context that characterise human artistic expression, leading such outputs to be perceived as less authentic or less “human” [3]. Beyond these concerns, researchers also note that the growing use of AI as a co-creative partner may reduce human agency by shifting creative control toward algorithmic processes [1], raising questions about how much - and which aspects of - creative decision-making must remain human in an increasingly AI-dominated landscape in order to preserve the experiential depth that underpins human art [3].

This tension caused by AI’s integration into creative fields has been extensively studied and is a defined research area by the name “human-computer co-creativity”. Human-computer co-creativity is defined by Davis [13] as a mode of interaction between human and system which is iterative and mutually informing, with neither party fully determining the creative output. Kantosalo & Toivonen [14] further distinguish this definition into two categories, alternating co-creativity, and task-divided co-creativity, and state the structure of interaction itself shapes how the creative agency is distributed. Shneiderman [15] additionally frames this challenge in terms of creativity support tools, arguing that AI-centred creative tools should expand rather than replace human creative capacity, and states the prioritisation of exploration and user-control over automation as a necessity to human-computer co-creativity. These discussed studies suggest that preserving creative agency isn’t just about how much of AI is used in the creative process, but how the interaction model is designed and implemented. These frameworks directly motivate the interaction model developed in this project, in which the AI functions as a supportive tool within the composer’s workflow rather than a dominant agent.

Current AI music tools are powerful, but they are often over-automated and tend to prioritise the simplification of the creative process over preserving human agency within it. Examples of this over-automation can be seen in systems such as AIVA, MusicLM, and OpenAI's Jukebox [4, 5]. AIVA generates complete original compositions from a style or mood prompt alone, presenting the user fully formed outputs with no ability to intervene in the process itself. MusicLM similarly accepts a natural language description as its sole input and returns a complete audio track, again with no creative decision available to the user. OpenAI's jukebox takes this approach further, generating raw audio including vocals and instrumentation from genre and artist prompts, with creative control similarly limited. These systems are classified as generative by Herremans functional taxonomy of music generation systems [6] and all shift creative agency entirely away from the user. There are systems like Magenta Studio [16] which offer greater interactivity by proposing musical continuations within existing compositions, however Magenta similarly provide no harmonic or theoretical context for its suggestions, limiting the composer's ability to learn from or critically evaluate the AI's output. This high degree of automation can reduce the composer's agency by shifting creative decision-making towards the model, and the lack of interpretability and transparency in the model's decisions limits the amount of knowledge and experience the user can obtain from the tool. Such systems risk positioning the composer as a passive recipient of AI-generated material rather than an active creative participant. These limitations highlight the need for a transparent, supportive, and non-intrusive AI tool that can enhance the compositional process without diminishing the user's creative control or understanding.

Therefore, the aim of this project is to design and develop an AI-powered music composition tool that enhances the creative process while preserving the user's agency over it. The system seeks to address the aforementioned limitations in existing AI music tools through three key mechanisms. Firstly, it aims to improve transparency and reduce over-automation through the implementation of a dual-model system. This system comprises a primary generative model, which produces a wide array of chord suggestions based on a given sequence, and a secondary explanatory model, which exists to clarify the reasoning behind the primary model's outputs. By providing the composer with a broader set of creative directions alongside clear, musically grounded explanations of these choices, the project aims to create a form of choice proliferation—expanding the number of creative decisions in a way that enables users to produce unique, personalised compositions. Secondly, the project aims to strengthen the user's control over the composition by implementing a hierarchical song model, allowing

greater control over tempo, time signature, and key for each section of the piece, as well as enabling efficient deletion, reordering, and addition of musical components. Thirdly, the project seeks to design and implement a streamlined user interface that integrates the dual-model system in a non-intrusive manner, ensuring that the AI functions as a supportive tool within the creative workflow rather than a dominant co-creator.

2. Requirements Analysis

Three primary aims were established at the outset of the project

A1: Implement a dual-model AI system providing transparent, explainable, and unique chord suggestions

A2: Implement a hierarchical song model giving the user fine-grained compositional control

A3: Design a streamlined UI that integrates the AI non-intrusively into the user's creative workflow

These aims represent the three key mechanisms which this project theorises will allow creative control to transfer from the AI to the user. Once the software is complete, these aims will be used to evaluate to what extent the goal of the project was achieved. From these aims, the following functional and non-functional requirements were derived.

Functional Requirements

FR1: The system shall allow users to create chord sequences with the use of AI-generated suggestions.

FR2: The system shall allow users to edit pre-existing chord sequences using AI-generated suggestions.

FR3: The system shall represent songs with four main tiers: song > section > sequence > chord.

FR4: The system shall allow users to navigate through and edit at each tier.

FR5: The system shall display a window providing a music theory explanation for the AI model's outputs.

FR6: The system shall allow users to preview and listen to their compositions using MIDI instruments.

-
- FR7:** The system shall allow users to change the tempo and rhythm of their project.
- FR8:** The system shall allow users to create, save, and open project files.
- FR9:** The system shall allow users to export their compositions as MIDI files.
- FR10:** The system shall include an archive feature where users can store unused sections of their compositions.

Non-Functional Requirements

- NFR1:** The system shall feature an easy to navigate user interface.
- NFR2:** The system shall generate chord suggestions within 3 seconds of the user's input.
- NFR3:** The system shall allow playback of MIDI compositions without noticeable lag
- NFR4:** The system shall autosave the user's project every 5 minutes to prevent data loss
- NFR5:** The system shall recover from system crashes by restoring the most recent autosave.
- NFR6:** The system shall store all user-created project files securely on the local device
- NFR7:** The system shall use version control to track progress (GITHUB)
- NFR8:** The system shall support future expansion and updates.

These requirements, along with the aims, will be referenced throughout the paper, especially in the evaluation where they will be used to measure the success of the final software.

3. Design & Implementation

The software - named "CeCe" after the patron saint of music Saint Cecilia - was developed as a desktop application using Electron and React for the frontend and Python for the backend AI system. Development followed an iterative approach, with components designed and implemented in sequence, each informing the next. This section covers the design and implementation of each major component of the system, showing in what order things were implemented, and the reason why certain design decisions were made.

3a. System Architecture Design

One of the earliest and most important design decisions in CeCe’s development was the choice of system architecture. A key concern from the outset was satisfying **NFR8**, which states that the software must support future expansion and updates. To satisfy this, the chosen architecture needed to ensure the decoupling between the business logic of the application and the user interface, so that extensions to one would not necessitate modification to the other. To achieve this CeCe adopts a four-layer architecture inspired by Fowler’s modular React architecture [7], which argues that treating React as a framework rather than a library leads to business logic becoming tightly coupled to UI components, increasing complexity and reducing testability. Fowler suggests a model consisting of a View layer, a State Management layer, a Domain layer, and a Data/Network layer, as illustrated in **Figure 1**.

CeCe’s View layer consists of purely presentational, stateless React components which get their state from hooks rather than handling them internally. This view architecture was chosen as it allows React components to stay decoupled from the software’s business logic, with communications instead being handled by the State Management layer, making the UI easier to test and extend independently. The State Management layer contains a global store, which exposes a mutable state, read-only selectors and callable actions to the View layer. This store is split into slices, each pertaining to a specific area of the software, and owning a state, a set of actions, and a set of selectors. This design choice further satisfies **NFR8** by allowing new features to be added as new slices without touching existing ones. State updates are propagated to the View layer through a Publish/Subscribe model, whereby React components subscribe to the global store and are notified automatically when relevant changes occur, triggering a re-render without components needing to check for updates. A direct benefit of this design was that it allowed the State Management layer to be developed independently, before the View or Domain layers existed, enabling the structure of the software to be defined before logic or UI. The Domain layer is comprised of pure logic. It has no knowledge of UI or state, it only receives inputs, applies logic, and returns results. The Domain - which is also split into slices – contains functions of two types, helpers and rules. These functions are fed into actions by the global store and called by the State Management layer, which then commits its results into the state. Finally, the Data/Network layer acts as the sole point of external communication. In CeCe the Data/Network layer is in charge of retrieving outputs from the Primary and Secondary models, as well as the saving and loading of project files. The Data/Network Layer also acts as a wrapper for libraries and APIs, meaning if external

dependencies change, it does not necessitate changes to the rest of the application. The further design and implementation of each of these four layers are discussed in future subsections.

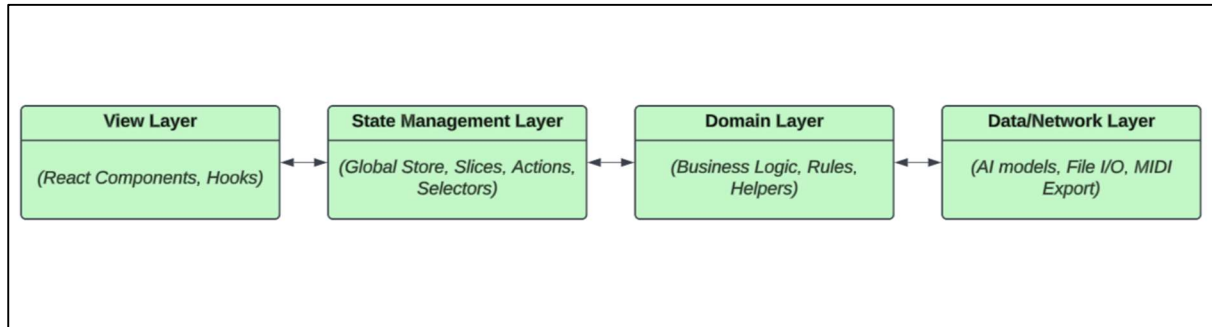


Figure 1: CeCe's Four Layer Architecture

3b. Implementation of the Global Store

Once the structure of the State Management layer was implemented and the slices defined, the implementation of the global Store began. Rather than adopting existing state management solutions such as Redux [11] and MobX [12], a custom store was implemented. This decision was made as research into these existing libraries showed incompatibility with CeCe's system architecture. Redux requires actions to be plain dispatched objects; this would hinder direct domain layer integration, and MobX's observable proxy wrapping of state objects would introduce complexity with CeCe's Domain Layer, which expects plain JavaScript objects as inputs. A custom implementation allowed the store to be designed specifically around CeCe's layered architecture, more specifically it allowed the implementation of bound actions seen in **Figure 2** and discussed below.

The implementation of Bound Actions allows references to the state, domain and other actions to be automatically injected into each individual action. The bound actions pattern works by wrapping each raw action in a higher-order function at store creation time. When an action is called, the wrapper then automatically injects the current state, domain layer, and other actions as parameters, meaning any part of the application can call an action without needing direct access to the store's internals. The benefit of this is it keeps the View and Domain layers decoupled from the store, whilst allowing actions to orchestrate complex operations across both layers. **Figure 3** shows how actions extract the information from the wrapper.

Additionally, a version counter is incremented and the publisher of the pub/sub model is called after each action call, ensuring React detects state changes even when no underlying data has been modified, thus preventing stale renders. With the global store established and implemented, development moved onto the implementation of the window system.

3c. The Window System

The window system was chosen as the next focus of development as its implementation would provide an early opportunity to test the interaction between the View and State Management layer, validating architectural decisions made in previous sections before the more complex logic was introduced. First the window system was implemented into the State Management layer. This process started with defining the life-cycle of the window; a transition function was then written to encode these states, as seen in **Figure 4**,

The transition function follows a finite state machine (FSM) pattern, in which the application state, current window ID and event are taken as parameters, and the new state of the window is returned. This pattern was chosen as window behaviour is inherently discrete, with each window occupying exactly one state at any given time and only a defined set of valid transitions between them. The states represent the different forms the window can take in the software, and consist of normal, moving, resizing, docked, undocking, maximised, and popped. Further explanations of what each of these states represent can be found below.

- Normal – Default resting state, window is stationary and interactive
- Moving – User is actively dragging the window by its title bar
- Resizing – User is actively dragging the window edge
- Docked – Window is snapped to the left, right, or bottom boundary of the screen
- Undocking – Transitional state between docking and moving, triggered when the user drags a docked window away from the boundary
- Maximised – Window fills the entire application surface, triggered by dragging to the top boundary
- Popped – Window has been detached into a separate browser window.

Next, window properties were defined in the State Management layer. Each window had a unique ID and the properties of size, position, and current state. Updating the size and position properties through View-triggered actions then required a three-stage drag system to be designed. The three stages consist of: commence drag — triggered when the user clicks the

window edge (for resizing) or the tab (for moving), and triggers the transition from normal to moving/resizing; update drag — which updates the targeted window properties based on mouse movement; and end drag — triggered when the user releases the mouse button, and again triggers the transition of the window state from moving/resizing to normal. This three-stage structure maps directly onto the browser's mousedown, mousemove, and mouseup events, ensuring drag state persists across multiple movement events between the initial click and release. Manipulation of z-order (the visual order the windows appear, top to bottom) required the creation of an ordered-list within the window state slice, this list held all visible window IDs and the order they appear, and an action was created which pushed the focused window's ID to the top of the list.

Window docking and maximising extended the drag system to support snapping behaviour, whereby windows being moved would transition to a docked state when the mouse was dragged to the defined bottom, left, or right boundaries of the screen, and would transition to maximised if dragged to the defined top boundary. Left, right, and bottom docks were defined in the window state slice and had the properties of width and contents — a list of window IDs currently occupying the dock; and maximising set the window size to match the size of the application window.

Finally, the ability for windows to be turned into separate browser windows was implemented, a feature that will be referenced as the "popping" and "unpopping" of windows. The decision to implement this feature was informed by its presence in several established compositional tools — such as Ableton, FL Studio, and Logic Pro — where the ability to detach panels into independent windows was seen to support multi-monitor systems and allow composers more freedom in the arrangement of their workspace. Replicating this behaviour in CeCe was therefore identified as a meaningful contribution to the tool's usability. As the creation and management of browser windows is handled in Electron's main process — outside of the renderer — APIs were created which could communicate between the renderer and Electron and trigger the creation of windows. However, the project at this point ran into several problems. As the global store existed in the main renderer, the popout windows could not access it directly. Instead, a secondary shared store needed to be created which — through a pub/sub communication model — would mirror the global store, and through actions — dispatched through APIs to the main renderer — would be able to trigger state changes. This system added the risk of the two stores becoming desynchronised, risking the single-state nature of the software, and also meant all windows which could be popped out would need to

be implemented differently from those which could not. After much trial and error the two-store system was finally implemented; however the added complexity it created in the implementation of windows led to the decision to exclude the feature from the final software.

The window system was an important feature as it allowed separate features such as the chord workshop and the music theory explainer to exist in their own space, giving the user control over their workspace layout in a manner consistent with professional compositional tools. Beyond usability, its early implementation served to validate the interaction between View layer – whose implementation will be discussed in the following section; and the State Management layer.

3d. Designing and Implementing the View Layer

The View layer of CeCe has a hierarchical architecture consisting of several components with differing responsibilities. The hierarchy flows from the App Shell — which houses constant UI elements such as the Top Bar and Desktop Surface — down through the Window Manager, which renders and layers Window Frame components in z-order. These Window Frames independently handle drag, resize, and refocus interactions for a given window, and contain a Window Content Host component where the window's content is visually displayed. The App Shell is rendered directly after the creation of the global store, which is then passed to the App Shell where it propagates through the hierarchy so that every component has access to the singular store reference. A visual representation of this hierarchy can be seen in Figure 5.

This separation ensured that feature components such as Song Editor and Chord Workshop remained decoupled from window management concerns, simplifying their implementation and making the addition of new window content straightforward, further satisfying **NFR8**. The implementation of the View layer architecture represents completion of the foundation of the software. The next stage of development will focus on satisfying project-specific requirements, starting with the development of the Primary Model

3e. Development of the Primary Model

The initial plan for this Primary model was to design and train a custom multi-headed transformer model – chosen as it is able to detect multiple relationships in patterns, which is useful in chord predicting - on the Chordinomicon [9] dataset, which contained 666,000 unique songs spanning a wide array of genres. The choice to create a custom model stemmed from two ideas, firstly the idea that simply integrating pre-trained AI models into a composition tool

was not complex enough a task for this project, and secondly that it would allow the software to implement a model which was custom designed and trained to fit its specific aims.

However, during data preparation, it became clear that Chordinomicon lacked structural information beyond individual chord sequences. In order for the training of a custom model to be warranted, it needed to provide insight on the relationship between different sequences and sections, not just the sequences themselves, and as it was discovered that Chordinomicon did not contain the necessary information to identify this structural relationship, a different dataset would be needed. This setback prompted a reassessment of the overall approach. Further research into other well-known datasets revealed that custom transformer models trained on these datasets were already well established in literature [8], and that a model trained on the same datasets would likely produce the same, if not inferior results to the existing implementations. Rather than investing significant development time into recreating a weaker version of existing models, it was decided that a pre-trained model would be used as the foundation, with a custom suggestion algorithm built on top of its output. This pivot meant that CeCe's original contribution would lie not in the model itself, but in the implementation and the data extracting methods of the model instead.

Efforts were therefore shifted to finding a pre-trained model which could satisfy four requirements. Firstly the model needed to be free to use and open source for research projects. Secondly the model needed to work off chord tokens which exposed at a minimum 3 qualities, root, chord quality, and bass note. This would allow the model greater pattern recognition and make it more robust to uncommon chords. Thirdly it needed satisfy **NFR2** speed requirement. Finally the model needed to be manipulable, or in other words allow direct access to the inner functions so that custom algorithms could be used to extract the needed data. After extensive search a model named MusicLang [10] was identified which satisfied all of these requirements. The implementation, limitations, and challenges in implementing this model will be documented in later sub-sections.

3f. Implementing the Hierarchical Song Model

The hierarchical song model represents the core data structure of CeCe, defining how a song is stored and manipulated within the application. A hierarchical structure was adopted as it reflects the natural organisation of music – a song consists of sections, sections consist of sequences, and sequences consist of chords; storing the structure like this allows for greater manipulation at every level of the composition. The structure of each song object – song

(project), section, sequence, and chord – is defined inside the Domain layer as a class. The class diagram showing the relationships, properties, and functions of the hierarchy can be seen in **Figure 6**.

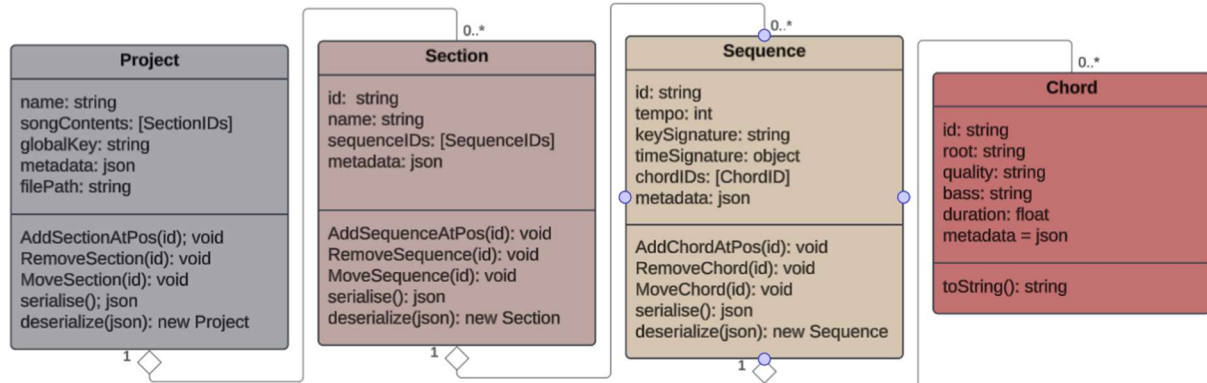


Figure 6: Class Diagram of Hierarchical Song Model

Each level of the hierarchy references its children through IDs rather than direct object nesting, and song objects can then be looked up by their ID using a dictionary stored in the application state. This decision was made to avoid deep object cloning, which in turn makes duplicating, deleting and moving objects easier within their parent. Each class also utilises `serialise()` and `deserialise()` functions, which allows the object to move from its read-only JSON representation in the State to its read/write object representation in the Domain, further aiding the decoupling of State and Domain. The implementation of this model directly satisfies **FR3** and goes towards satisfying **A2**.

A challenge during development was converting the hierarchical song model — which consists of song objects that do not know their chronological position — into a format in which every object knows its position within the composition. To address this, a Song Space was designed: a chronologically oriented representation of the song consisting of two dictionaries, beats and objects. The beats dictionary mapped beat positions to their assigned tempo, time signature, and key (obtained from the parent Sequence), whilst the objects dictionary mapped object IDs to their type, start beat, and end beat. An algorithm was then written to extract the song's contents from the application state and convert it into the Song Space, allowing the software to visually represent the composition at each level of abstraction within the View layer. To ensure that the Song Space was always in sync with the application state, a song version counter — consistent with the versioning pattern established in the State Management

layer — was implemented, incrementing each time a change to any song object was made and automatically triggering a recalculation of the Song Space.

The Song Editor was then implemented as the primary composition view of CeCe, providing a visual timeline representation of the song's contents. The Song Space, described in the previous section, served as the data source for this view — its chronologically ordered beat and object dictionaries were used to render song objects across a timeline at two levels of abstraction: Section View, which displayed the song at the section level (intro, outro, etc), and Sequence View, which displayed the song at the sequence level (chord sequences). To support the repositioning of objects within the timeline, the three-stage drag system established in the Window System was reused, with commence, update, and end drag actions adapted to update the start and end beats of the targeted object rather than window position. Basic object manipulation was also implemented, including duplication and deletion of sections and sequences, directly satisfying **FR4**.

3g. MIDI System Implementation

The MIDI system was an essential part of CeCe's development as it allowed the hierarchical song model to transition from a purely visual representation to something the user could interact with audibly. The requirements referencing MIDI — **FR6**, **FR9**, and **NFR3** — state that the system must be able to play back the user's composition with minimal lag and allow them to export it as a MIDI file. The first step in implementation was converting the Song Space into playable MIDI events. The pipeline - illustrated in **Figure 7** - begins by cycling through the Song Space beat by beat, detecting when the chord changes. Each change produces a chord event containing the chord's root, quality, bass, start beat, and duration in beats, resulting in an array of chord events which capture a chronological view of the composition at the chord level. Each chord event's musical description — root, quality, and bass — is then converted into MIDI note numbers using a harmony module, representing the actual notes of the chord. These note events are then converted into note On and note Off events with precise millisecond timestamps, using a secondsAtBeat lookup to correctly account for tempo changes across the composition, and a rhythm pattern to distribute notes across each beat rather than playing them as static blocks. All events are then merged into a single list and sorted by time, producing a global MIDI event list ready for playback.

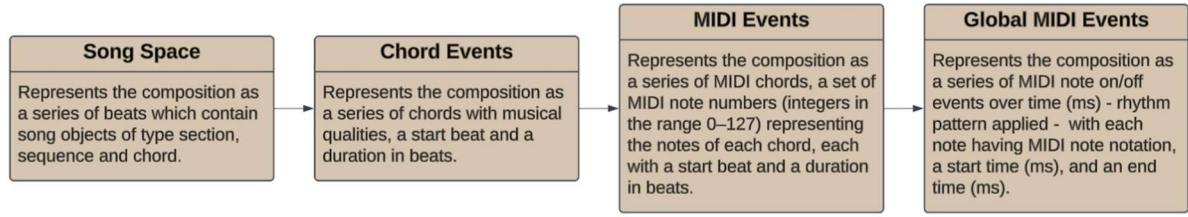


Figure 7: Diagram of Global MIDI Event Pipeline

MIDI playback was implemented using the Soundfont-Player library [17], chosen for its collection of over 100 built-in MIDI instruments. A MIDI interface was implemented to initialise the player and schedule note on/off events. To satisfy **NFR3**, which requires playback with no noticeable lag, a global audio context was implemented to provide a consistent clock governing timing across the entire application, ensuring MIDI playback remained unaffected by system load times. The introduction of a global audio context also ensured that the View's player header and the MIDI scheduler remained permanently in sync. Finally MIDI actions were implemented into the State Management layer to control the playing, stopping and moving of the MIDI player, and a simple menu was added to allow the changing of MIDI instrument and MIDI rhythm.

3h. Implementing the Primary Model

With MusicLang now identified as CeCe's primary model, as discussed in the "Development of the Primary Model" section, implementation began with integrating the model into CeCe's Data/Network layer. MusicLang is a GPT-2 based chord prediction model which operates on tokenised chord sequences, taking a series of chord tokens as input and predicting the most likely continuation. MusicLang was implemented into the Python backend of CeCe, and exposed to the Data/Network layer by an Electron API. An interface was then created inside the Data/Network which contained two functions, one which generated chord suggestions for an empty sequence in a given key signature, and one that generated chord suggestions based on a given sequence and key signature.

Since the MusicLang model was specifically designed to generate full chord sequences [10], a custom algorithm had to be applied to its output to generate a diverse range of chord suggestions. The model splits the chord into 6 chord tokens, titled `CHORD_CHANGE`, `CHORD_DEGREE`, `TONALITY_DEGREE`, `TONALITY_MODE`, `CHORD_EXTENSION`, and `CHORD_OCTAVE`. While this level of tokenisation allowed for deeper pattern recognition in the sequences, it somewhat limited the types of chords that the model could

accept and produce, this limitation of chord representation would be inherited by CeCe, as the software could not contain any chords which were unrepresentable in the model. Weighing the pros and cons, the decision was made to continue with the model as the range of available chords still had enough diversity to allow the creation of unique and interesting chord sequences. With this in mind, a custom tokenizer and detokenizer were designed to bridge the model's token format and CeCe's chord representation.

Testing revealed that MusicLang could generate a single chord suggestion in approximately 15ms; factoring in an API communication overhead of approximately 0.5 seconds, this meant that 160 chord suggestions could be generated within approximately 3 seconds, allowing the system to offer a diverse selection of chords whilst satisfying NFR2's generation speed requirement. To transmit the input sequence to the primary model over the API, a domain-specific language was created which compactly described the key signature and chord contents of the sequence. Full chords were extracted from the model by first tokenising the input sequence, then predicting the chord one token at a time by applying temperature-scaled sampling to the raw logits, using position-masking to get only the desired token types. This process was repeated for each of the six tokens until a full chord was produced. The probability of this chord was then calculated by combining the log probabilities of each token. In order to extract a wide array of diverse chords from the model, suggestion generation was split into four batches, each with differing temperature levels, from a temperature of 0.7 which generated standard continuations of the chord sequence, to a temperature of 1.6 which generated harmonically less conventional chords. Each batch generated forty chord predictions, and lower temperature batches were given a higher weight to account for the differing probability scales across temperature settings. A code snippet showing this entire algorithm can be seen in **Figure 8**. The predictions and their probabilities were then deduplicated and sorted by probability, before being returned through the API to the Data/Network layer for processing. The effectiveness of this algorithm will be evaluated later in the Testing and Evaluation section.

3i. Designing & Implementing the Chord Workshop

The Chord Workshop handles the editing and creation of chord sequences within CeCe. To accommodate this, a new window was created, building upon the previously implemented window system. Since the Chord Workshop displays its content in a similar way to the Song Editor, this similarity allowed many components to be reused with slight modifications. Components

including the Song Space structure, MIDI player, object dragging system, and delete and duplicate functions were all adapted and reused within the Chord Workshop. All of this was implemented in a custom Chord Workshop slice in order to keep it decoupled from the Song Editor. With a sequence of chords now displayable and editable, implementation of the primary model into the workshop began, starting with the suggestion generation system. For this, an "Add Chord" button was created that appears when the user hovers over a gap between chord blocks, or at the end of the sequence. When this button is pressed, a new chord object is created called "suggested chord," which is initially set to null. The creation of this block triggers communication with the backend to gather the suggested chords. While this suggested chord block is empty, it takes up space in the Chord Workshop, and once a chord is chosen by the user it can be previewed. This is significant as it allows the user to try out a variety of chords before confirming their selection by pressing the tick button, satisfying **FR1**, **FR2**, and **FR6**.

The chord suggestions are shown in the Chord Workshop in a side tab to the right, and have four categories — common, uncommon, strange, and inadvisable — divided by applying three empirically determined probability thresholds to the suggestions generated by the primary model. These thresholds were determined through iterative testing across multiple sequences of varying length and key, and the goal was to produce meaningful spread across all four categories. Early threshold values were shown to produce too many common suggestions and almost nothing in the later categories. After iterative testing, a thresholding system was established in which probabilities were normalised and were split into top 25% (common), 25% to 50% (uncommon), 50% to 75% (strange) and bottom 25% (inadvisable), seen in **Figure 17**. Additionally, a text input is provided allowing the user to enter a custom chord. The Chord Workshop is also where the user can change the tempo, time signature, and key signature of the sequence, achieved through dropdown menus for the time and key signature, and a text input for the tempo, satisfying **FR7**. Here the user can also change the name of the section in which the sequence is held.

The Chord Workshop's suggestions appearing in a side tab was a deliberate decision made to keep suggestions visible without obscuring the workshop space. A separate window was considered for the chord suggestions, but this was rejected as it would interrupt the user's workflow by drawing the user's attention from the workshop. The side panel keeps the suggestions constantly visible without forcing any additional user action to see them, this design decision directly supports **A3** as it allows the AI suggestions to be present without being intrusive. Similarly, the design of the suggestion preview and confirmation feature allows the suggestions

chord block to immediately occupy space in the timeline, giving the user a visual representation of where the chord will sit before committing, clicking any suggestion in the side panel immediately makes it occupy the suggestion block and allows the user to preview it audibly within the context of the surrounding sequence. This feature also ensures that no chord/creative-decision is ever added to the composition without an explicit tick button press from the user.

As creating a new MIDI scheduler was deemed not time-efficient, the scheduler used in the Song Editor was altered to allow the Chord Workshop to use it as well, switching out the global Song Space for its sequence-specific Song Space. The only side effect of sharing a scheduler is that the Chord Workshop and Song Editor cannot play at the same time, which in practice benefits the system by preventing audio conflicts between the two features. A screenshot of the completed Chord Workshop can be seen in **Figure 9**.

The Chord Workshop represents the primary point of interaction between the user and CeCe's AI system. By requiring explicit user confirmation before any chord is added, and presenting suggestions non-intrusively within the composer's workflow, it represents the principal mechanism through which **A1** and **A3** are delivered.

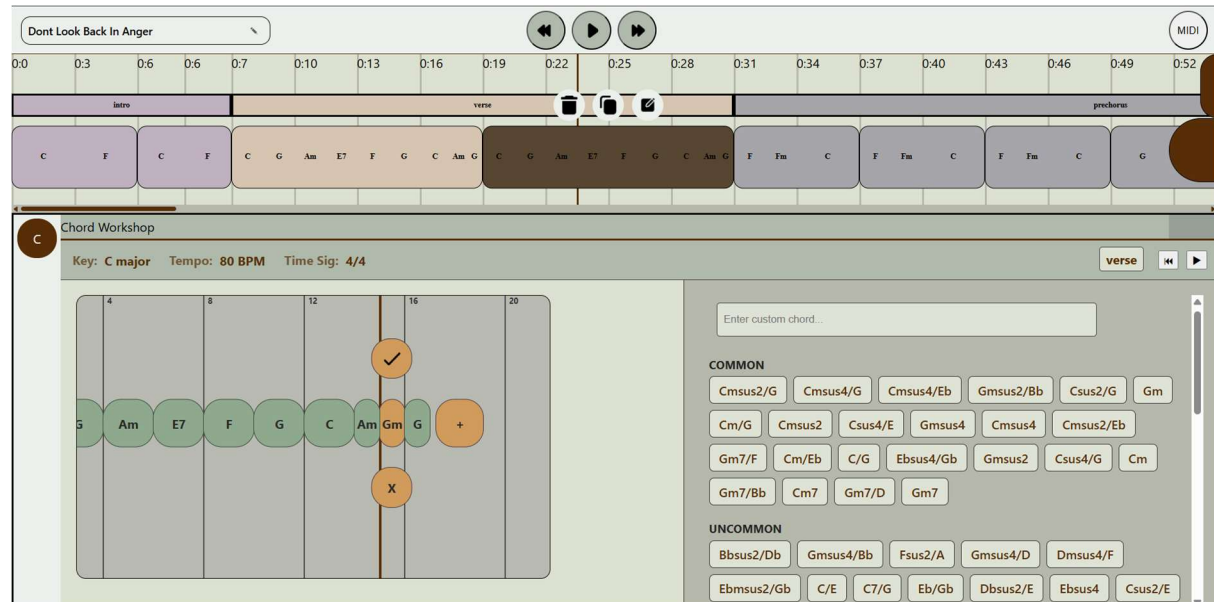


Figure 9: Screenshot from finalised software of the Chord Workshop

3j. The Secondary Model

The secondary model serves as the explanatory component of CeCe’s dual-model AI system, providing music theory context for the chord suggestions produced by the primary model, directly supporting **A1** and **FR5**. Before development began, Vicuna was identified as a suitable candidate due to its instruction-following capabilities, small model size, and ability to run locally on the user’s device. At the time the decision to implement a local model was made to firstly allow for free access to an open-source language model, and secondly to allow the software to run without internet connection. Vicuna was implemented into CeCe’s Python backend and exposed to the Data/Network layer through Electron’s APIs, following the same interface and communication pattern established for the primary model.

With the model implemented, prompt engineering began to try and extract both expressive analysis and harmonic analysis of chord sequences. However it was during this process that issues became apparent regarding the speed of the Vicuna model. From the request of the Data/Network layer, to the receiving of results, the software took ~50s, this was a problem as in order for the secondary model to achieve its function, it needed to give fast feedback on developments of the composition. As Vicuna offers a range of models of varying size, a smaller model was implemented, however this model still took ~40s to produce results. Efforts were then turned towards optimising the model, first reducing the context window from 1024 tokens to 128, this would allow the model to maintain context over 300 words including prompt and response, enough for the secondary models task. Next experiments with Vicuna models of varying size were carried out, the smallest models were unable to produce useful results, and larger models took too much time, in the end a middle ground was found in the Q3_K_S, a mid-sized model. While these reduced time load time to ~26s, this was still too large for the task.

On top of these problems, both the harmonic and affective analysis produced by the model were inconsistent and inaccurate, hallucinating music theory explanations and relationships within the sequence that did not exist. An inverse relationship between the performance of the model and the speed was identified, with more accurate answers taking substantially longer to generate. In order to address these large waiting times, a pre-loading feature was implemented in which music explanations of chord sequences were generated on the creation or editing of the sequence. This would allow the explanations to appear instantaneous to the user while maintaining a large enough model to generate satisfactory answers. However, with this implemented, the explanations generated by the secondary model were still unreliable and

unsatisfactory, so it was ultimately deemed unfit for inclusion in the final software. Further evaluation of the secondary model's performance can be seen in the evaluation section.

3k. File Management System

Finally, the file management system was implemented; this feature was important as it would turn CeCe from a hypothetical compositional project tool into a continuous software that allows the user to save and load projects, thus satisfying FR8. The first step of implementation was defining what a project file should look like. Only project-specific information such as composition shape and contents, project names, project file destination, and any necessary metadata needed to be saved, as the rest of the state was loaded fresh every time. This meant that the project file only needed to store four slices of state — Project, Section, Sequence, and Chord — in order to save and load full compositions. These slices were then compiled into a single JSON file and given the custom file type "CECEP", standing for "CeCe Project". A new interface was defined in the Data/Network layer that extracted copies of these four slices, compiled them into the JSON file, and then ran validation checks to ensure it was capable of being loaded. An Electron API was then set up to facilitate communication with the main Electron process, where a file browser would be opened, allowing the user to select their desired destination — a destination that would then be saved within the Project slice to allow the user to save the project later without having to reselect it. An error system was set up via the API, which would trigger in cases where the user loads an invalid file, renames their project to an invalid file name, or attempts to save a project with no destination. Finally, a two-way naming system was implemented, which allows the user to change the project name by changing the file name, and vice versa.

3l. Refining the User Interface

During development, a component class system was used to apply CSS properties to each View layer component, this made modification of the user interface easy and allowed for its refining throughout the project. This section touches on design decisions related to the user-interface that were not mentioned in previous sections.

It was decided that CeCe would incorporate a lighter colour scheme for several separate reasons. Firstly lighter colour schemes produce a higher contrast ratio between body and text, which significantly reduces eye strain from long periods of usage. Secondly research into existing compositional tools showed a large number of them – such as Sibelius, Finale, and

MuseScore – feature lighter colour schemes, whereas mixing/mastering DAW software – such as Logic, Ableton, and Reaper – often by contrast have darker colour schemes. CeCe is a compositional tool, so the implementation of a light colour scheme would allow it to visually align with compositional software as opposed to the often darker DAWs. Thirdly it was decided that a lighter colour scheme would make the software seem less intrusive with its AI suggestions, it was reasoned that that dark interfaces tend to feel more dominant, and while this may work for other software, A3 states the software should be non-intrusive and aid the creative process. The implementation of a lighter, more contrasted colour scheme allows CeCe to feel visually less-assertive and more usable.

The colour pallet chosen, illustrated in **Figure 18**, consists of calm, muted colours, which aim to make the software feel fun and lively while still grounded. The muted tones were specifically chosen as they would make the software feel calm and un-intrusive, while limiting glare and ensuring the software is comfortable to look at for long periods of time. The specific colours chosen were informed by Mehta & Zhu’s study, which directly linked light, cool-toned colours to the creation of innovative and explorative spaces.



Figure 18: Shows the Primary Colours used in the Software

Contrast testing, illustrated in **Figure 19**, was also applied during development to ensure that the specific shades chosen for body and text provided a level of contrast that would not cause strain to the eyes, the testing was carried out on Adobe Colour Contrast Analyser [18] and showed the chosen colours passed all contrast tests

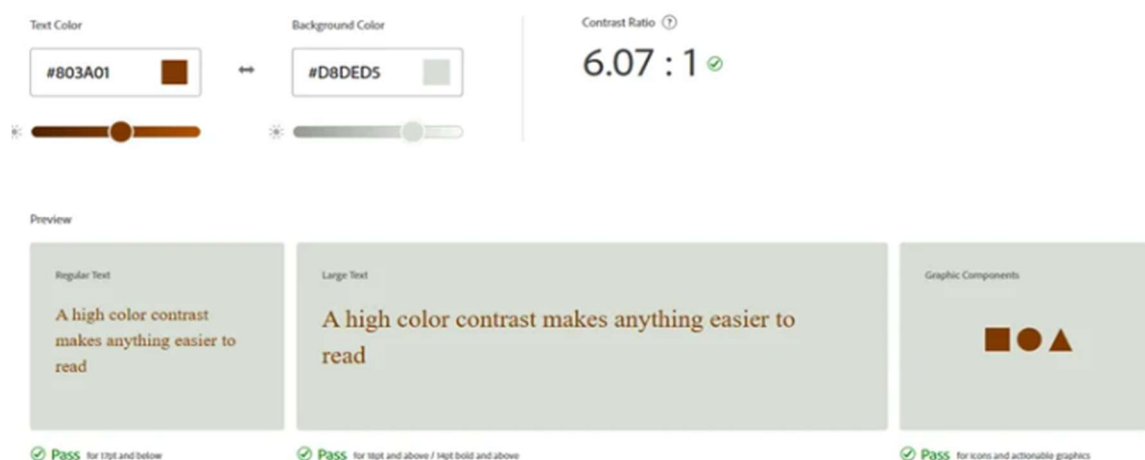


Figure 19: Shows screenshot of contrast testing from Adobe Colour Contrast Analyser

4. Evaluation & Testing

This chapter aims to evaluate the performance, functionality, and usability of CeCe against the requirements defined in Section 2. Both functional and non-functional requirements are tested systematically, followed by a deeper evaluation of the behaviour of the Primary and Secondary Models, the user interface, and the systems impact on creative agency. The aim of this evaluation is to determine the extent to which the system meets its intended goals and to identify any limitations or areas for further improvement.

4a. Functional Requirement Evaluation

Functional testing was conducted by verifying each Functional Requirement, FR1-FR10, against the implemented software. Each requirement was tested manually within the final build of CeCe, with results recorded as fully satisfied, partially satisfied, or not satisfied.

FR1 – *Fully Satisfied*

FR1 required that the system should allow users to create chord sequences with the use of AI-generated suggestions. This was tested by creating an empty section, opening it in the Chord Workshop, selecting a key signature, and requesting and selecting initial and subsequent chord suggestions from the MusicLang model. The system successfully generated suggestions and allowed the user to build a complete sequence, meaning **FR1** is fully satisfied.

FR2 – *Fully Satisfied*

FR2 required the system to allow users to edit and add to pre-existing chord sequences with use of AI-generated suggestions. This was tested by selecting an existing sequence object, opening it in the chord workshop, and requesting new suggestions from the MusicLang model to insert at various positions. The system allowed chords to be added, replaced, and modified within the sequence, meaning **FR2** is fully satisfied.

FR3 – *Fully Satisfied*

FR3 required the system to represent the composition in four main tiers – song, section, sequence, and chord. This was tested by creating a project containing multiple sections, each with several sequences and chords each, and inspecting the global store to confirm that each object correctly referenced its children through IDs. Serialising and reloading the project

preserved the same hierarchical structure, demonstrating that the representation was implemented correctly. **FR3** is therefore fully satisfied.

FR4 – Fully Satisfied

FR4 required that the system to allow users to navigate through and edit each tier of the compositional hierarchy. This was tested by loading a project containing multiple section, sequences, and chords, then editing sections and sequences in the song editor, and opening a sequence in the chord workshop and editing the chords within. The system allowed song objects to be added, removed, and modified at all hierarchical level, meaning **FR4** is fully satisfied.

FR5 – Partially Satisfied

FR5 required the system to provide a music-theory explanation for primary models chord suggestions. This was implemented using a secondary model based on Vicuna-7B, which was integrated into the backend of the software and tested. The system was able to produce explanations for some chord relationships, demonstrating partial functionality. However, testing showed that Vicuna frequently hallucinated harmonic relationships that did not exist, and generated vague and generic expressive analysis. Testing also revealed ~26s inference times per explanation, making it unsuitable for real-time use as the user requires immediate explanations. As a result, the explanations were not reliable enough to meet the requirements fully, and **FR5** is therefore only partially satisfied.

FR6 – Fully Satisfied

FR6 required the system to allow users to preview their compositions using MIDI playback. This was tested by loading a project containing multiple sections and sequences, initiating playback, moving the play head, and replaying the composition. Additional tests were performed by changing the selected MIDI instrument and rhythm through the MIDI settings menu. The system successfully played back the composition, responded correctly to play head movement, and updated playback according to the selected instrument and rhythm, meaning **FR6** is fully satisfied

FR7 – Fully Satisfied

FR7 required the system to allow users to change the tempo and rhythm of their project at different points in the composition. This was tested by loading a full project, selecting individual sequences, and modifying their tempo and rhythm values before initiating MIDI

playback. The Song Space updated these changes in real time, and the resulting playback reflected the new timing and rhythmic structure. **FR7** is therefore fully satisfied.

FR8 – Fully Satisfied

FR8 required the system to allow users to create, save, and open compositions as project files. This was tested by creating a project containing multiple sections, sequences, and chords, saving it to disk, restarting the software, and reloading the saved file. The project was then modified, saved again, closed, and reloaded to verify that updates persisted correctly. The system successfully saved, loaded, and restored all project data without corruption, meaning **FR8** is fully satisfied.

FR9 – Not Satisfied

FR9 required the system to export compositions as MIDI files. This feature was not implemented in the final version of the software due to time constraints and the prioritisation of core functionality. As a result, **FR9** is not satisfied.

FR10 – Not Satisfied

FR10 required the system to feature an archive feature which allowed users to store and use song objects. This feature was not implemented in the final version of the software due to time constraints and the prioritisation of core functionality. As a result, **FR10** is not satisfied.

Functional Requirements Overview

Overall seven out of 10 functional requirements were fully satisfied, one was partially satisfied, and two were not implemented due to time restraints.

Non-Functional Requirement Evaluation

Non-functional testing evaluated CeCe against performance, usability, reliability and maintainability requirements. Each requirement was tested using the final build of the software, with results recorded as fully satisfied, partially satisfied or not satisfied based on measurable evidence and observed behaviour.

NFR1 – Fully Satisfied

NFR1 required the system to feature an easy-to-navigate user interface. Usability testing was conducted with peers of varying musical and technical knowledge to assess clarity and navigability of the Song Editor and the Chord Workshop, the full results of which can be found

below in the user evaluation section. Participants on a whole were able to locate key features, understand hierarchical structure, and perform editing tasks with no prior knowledge or guidance. No major usability issues were identified, meaning **NFR1** is fully satisfied.

NFR2 – *Partly Satisfied*

NFR2 required the system to generate chord suggestions within 3 seconds of user input. This was tested by generating full generation sets across multiple runs under multiple conditions, the results of which can be seen in the primary model evaluation below. Average inference time was approximately **5.6** seconds on the CPU, with results varying between 3 and 8 seconds, meaning **NFR2** is partly satisfied.

NFR3 – *Fully Satisfied*

NFR3 requires the system to allow real-time playback of MIDI compositions without noticeable lag. This was tested by initiating MIDI playback across a range of tempos and compositions of varying length. Playback remained stable, Player head movement and tempo changes updated immediately and playback produced no audible jitter or latency, meaning **NFR3** is fully satisfied.

NFR4 – *Not Satisfied*

NFR4 required the system to autosave the users project every 5 minutes to prevent substantial data loss during crashes. This feature was not satisfied due to time constraints and prioritising of core functionality. As a result, **NFR4** was not satisfied.

NFR5 – *Partially Satisfied*

NFR5 required the system to restore the most recent auto-save in the event of a system crash. Autosave was not implemented in the final build, however the manual save and load functionality was fully implemented and tested. This allowed users to recover the most recently saved version of their project following a restart, provided they had saved prior to the crash. As a result, the system offers partial recovery capability but does not meet the full requirement for autosaved crash recovery. **NFR5** is therefore partially satisfied.

NFR6 – *Fully Satisfied*

NFR6 required all user-created project files to be securely stored on the local device. To test this file operations were inspected to confirm that all project saves and loads occurred exclusively on the

user's local filesystem. No external network requests were made during project handling, meaning **NFR6** is fully satisfied.

NFR7 – *Partially Satisfied*

NFR7 required that version control should track the progress of development. The project repository was reviewed for commit history, branching and documentation. All early development were seen to be tracked through GitHub with clear and consistent commit structure, however later development showed a few instances of bulk commits and vague commit comments, meaning **NFR7** is only partially satisfied.

NFR8 – *Fully Satisfied*

NFR8 required the system to support future expansion and updates. The architecture was reviewed for modularity and extensibility. The implementation of the slice based Domain and State Management layers, the modular stateless View layer, and the decoupled Primary/Secondary Model interfaces support future enhancement such as improved AI models, additional editing tools, and expanded functionality. **NFR8** is therefore fully satisfied.

Non-Functional Requirements Overview

Across the eight non-functional requirements, four were fully satisfied, two were partially satisfied, and two were not satisfied. Overall, CeCe demonstrates strong usability, performance, reliability and extensibility characteristics.

4b. Evaluation & Testing of the Primary Model

One of the primary aims of the project (A1) was to implement a dual-model AI system providing transparent, explainable, and unique chord suggestions. The two descriptions – transparent and unique – directly depend on the performance of the primary model., with explainability being addressed by the secondary model. Transparency refers to the extent that the user can surmise and determine the model’s internal reasoning, while uniqueness concerns the diversity and originality of the chord suggestions it produces. Evaluating these properties requires examining both the behaviour of the model, and the mechanisms through which its decision-making can be exposed. Speed and correctness are also evaluated.

Primary Model Transparency Evaluation

The system aims to convert the model's internal probability distribution into a human-readable format by splitting the suggestions into four probability-based categories — Common, Uncommon, Strange, and Inadvisable. While this does not expose users to raw logits or log probabilities, it allows them to see that the model considers Common suggestions highly probable continuations, and Inadvisable ones low-probability outliers, giving them genuine insight into what the model thinks about their sequence. That said, there are limitations to the model's transparency. While the model exposes confidence level in an abstract form, it does not expose its musical reasoning. A user can see that a chord is considered inadvisable, but not specifically why. This is a constraint of the neural architecture, as the model searches for patterns without any knowledge of what those patterns musically represent. This caveat falls under the explainability of the model, which is the responsibility of the secondary model rather than the primary and is therefore not assessed here. Overall, while the model does not achieve full transparency, its generation of a wide array of suggestions and its sorting of those suggestions into a human-readable format contributes meaningfully to satisfying the transparency component of A1.

Primary Model Uniqueness Evaluation

Uniqueness of the chord sequences produced is a more difficult metric to measure and evaluate. For this evaluation, uniqueness is defined as the diversity and variety of musical elements - root notes, chord quality, and bass notes – present across the suggestions generated by the model. According to this definition, an algorithm was designed which would generate 160

suggestions for 5 sequences of varying key and length, the results of each illustrated in **Table 1**

Table 1: Results of generation tests of the primary model

Test Sequence	Suggestions	Root Notes	LP Min	LP Max
C major – 2 chords	98	11	-11.0	-2.8
C major – 4 chords	78	11	-9.8	-2.0
C major – 6 chords	63	11	-10.7	-1.2
A minor – 2 chords	88	12	-12.9	-2.7
A minor – 4 chords	90	12	-12.1	-2.7

The results from these tests were then used to create diagrams showing the distribution of root and quality in the 426 generated suggestions. These results – illustrated in **Figure 10** - demonstrate three distinct things. First, they demonstrate that the primary model has full chromatic and qualitative coverage, with every root and quality appearing in the generated test set. Secondly, they show that, while the model is more likely to suggest diatonic chords within the input key, it is also able to suggest borrowed chords from other keys — even the rarest chord roots, such as Gb and Db, still each represent over 1% of the overall suggestions generated. This is important as it shows that the model not only suggests standard chords within the given key, but can suggest chords the user may not have independently considered. Finally, the quality distribution across 426 suggestions spans seven distinct chord types, with the weighting towards minor-quality chords — comprising approximately 60% of output — reflecting the statistical prevalence of minor-quality degrees in Western diatonic scales. Rather than generating uniform randomness, the model produces calibrated variety, demonstrating that the suggestions produced are both contextually grounded and sufficiently diverse to satisfy the uniqueness criterion defined at the outset of this evaluation.

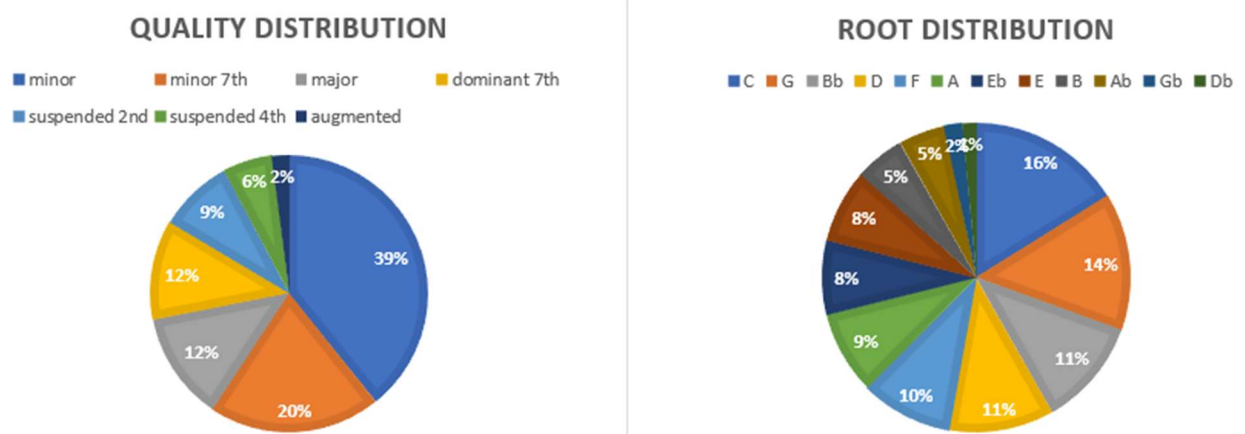


Figure 10: Pie Charts Showing the distribution of quality and root over the primary models suggestions

Additionally, a test was run to assess the effect sequence length has on the number of unique suggestions produced, the results of which are illustrated in Figure 11. The results show no clear trend between sequence length and the number of unique suggestions produced. It would be expected that a standard autoregressive model would exhibit a consistent inverse relationship, where an increase in sequence length causes a decrease in unique suggestions due to the model's growing contextual focus. However, because of the temperature-varying algorithm used to extract suggestions, the model is still able to produce a diverse variety of chords even with greater context. This data was then used to generate an estimate of the number of unique 12-chord sequences the model can generate, giving a result of 2.88×10^{19} unique chord sequence. While this is a rough approximation, it still demonstrates the possibility the primary model holds to help the user generate truly unique compositions.

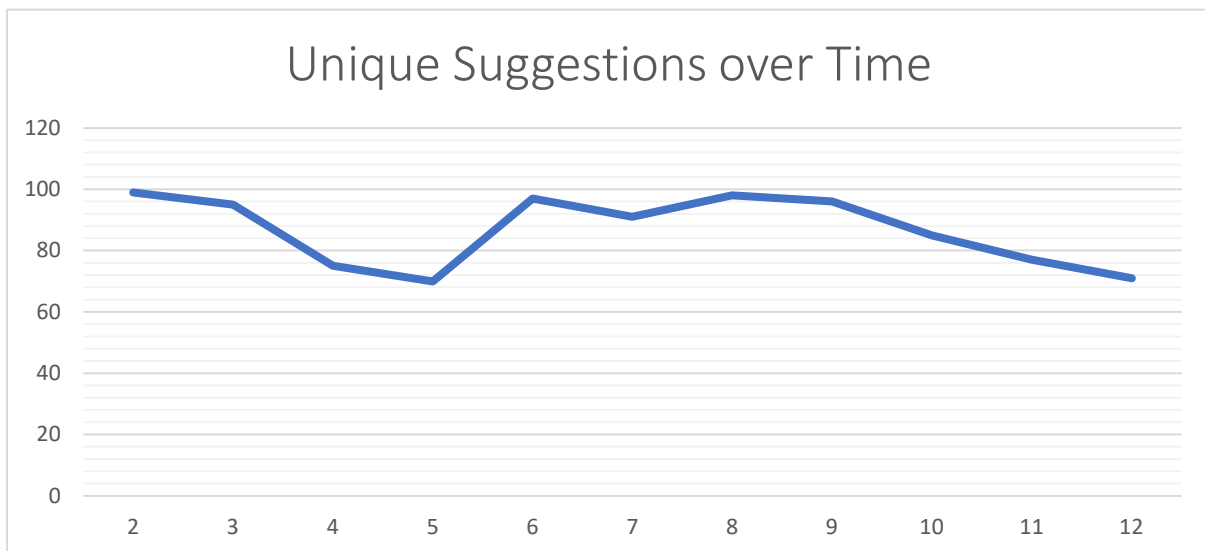


Figure 11

Primary Model Correctness Evaluation

While the model has shown the ability to generate an incredibly large number of sequences, the correctness of these sequences has not yet been fully evaluated. As music is a subjective field, it is difficult to measure the correctness of certain suggestions, as many would argue that there is no incorrect chord progression in music. However, for the sake of this project, correctness of the primary model is defined as the extent to which the model's output aligns with musical trends in Western music.

The results in **Figure 10** show that the distribution of roots and qualities align with patterns in real music theory. The root distribution shows C (16%) and G (14%) as the most frequent suggestions across both C major and A minor sequences. These are the tonic and

dominant of C major, and both are contextually prominent in A minor as well, the model was not told this, it learned to recognise the pattern from its training data. Bb (11%) and Eb (8%) are not diatonic to either tested key; they are borrowed chords from the parallel minor and are among the most commonly used borrowed chords in contemporary Western music, appearing consistently in pop, rock, and film music. Their prominence further demonstrates that the model is not suggesting random chords, but chords that musicians would genuinely use in their compositions.

On the lower end of the distribution, Gb (2%), Db (1%), and Ab (5%) appear with notably low frequency, reflecting their rarity in the context of the tested keys. The relative prominence of Ab over Gb and Db, Ab being a more commonly used borrowed chord, suggests the model has learned to reflect the nuances of harmonic distance in its output. The quality distribution reinforces this further: the dominance of minor-quality chords at 42%, the meaningful presence of dominant sevenths at 14%, and the low frequency of augmented chords at 2% all mirror the statistical proportions found in real Western tonal music, providing evidence that the model has learned musically correct harmonic conventions from its training data.

A full evaluation of correctness would require structured user testing, in which participants with musical knowledge would assess the musical appropriateness of the model's suggestions against a defined set of input sequences. This is not possible within the time constraints of this project, as outlined in the project ethics section. However, the observations made above provide sufficient evidence that the model's output aligns with the conventions of western tonal music to a satisfactory degree, therefore fulfilling the correction requirement for the scope of this project. Full user testing remains an area for future development.

Primary Model Speed Evaluation

To evaluate the model speed, a test was created that would measure the inference time over a wide range of input sequence lengths, the results of this test can be viewed in **Figure 12**. As you can see from the results, the primary model fails to satisfy **NFR2**. While the model speed does not satisfy the requirement, the results show that the model approaches the requirement frequently, and even the maximum inference values identified still produce the suggestions in satisfactory time. I will therefore declare **NFR2** as partly satisfied

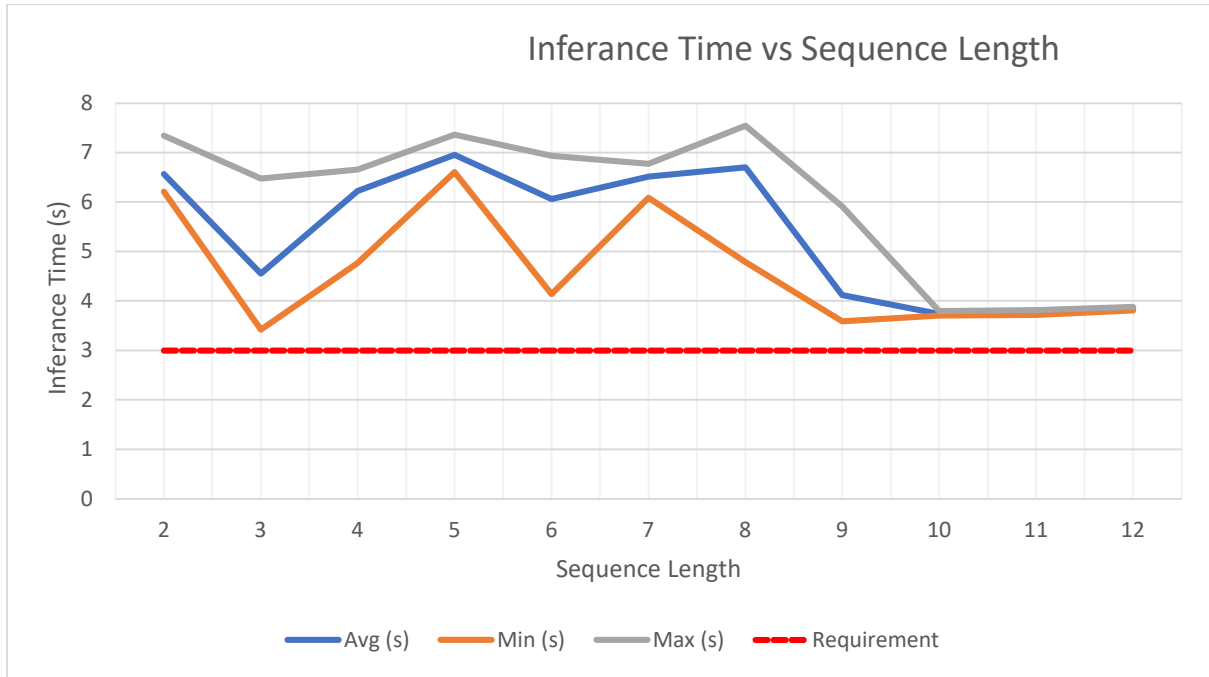


Figure 12

Overview of Primary Model Evaluation

Overall, tests and evaluation show that the model achieved three of the four requirements evaluated in this section. The model was shown to satisfy transparency, uniqueness, and correctness requirements, however failing at the speed requirement.

4c. Evaluation & Testing of the Secondary Model

The secondary model was intended to provide harmonic (technical) and affective (emotive) analysis of the sequences and suggestions produced by the primary model, and is responsible for satisfying the explainable requirement for AI. The secondary model that was implemented was not capable of satisfying it due to two performance issues relating to Speed, and Accuracy.

Secondary Model Speed Evaluation

Firstly, for the secondary model to be effective as a theory explaining tool, it needed to produce real-time explanations of suggestions generated by the primary model. A speed test conducted, the results of which are illustrated in **Figure 13**, show that the average inference time for harmonic analysis is 55.9 seconds and the average inference time for affective analysis is 58.0 seconds. This this time can be driven down to the ~26s, doing so makes the results even more accurate. While such a large inference time makes real-time use difficult, the implementation of pre-loading allowed the secondary model's explanations to be generated on the creation or editing of a sequence, while this still doesn't allow for real time primary model justification in

the Chord Workshop, it would allow for the user to view musical analysis of their compositions after the fact.

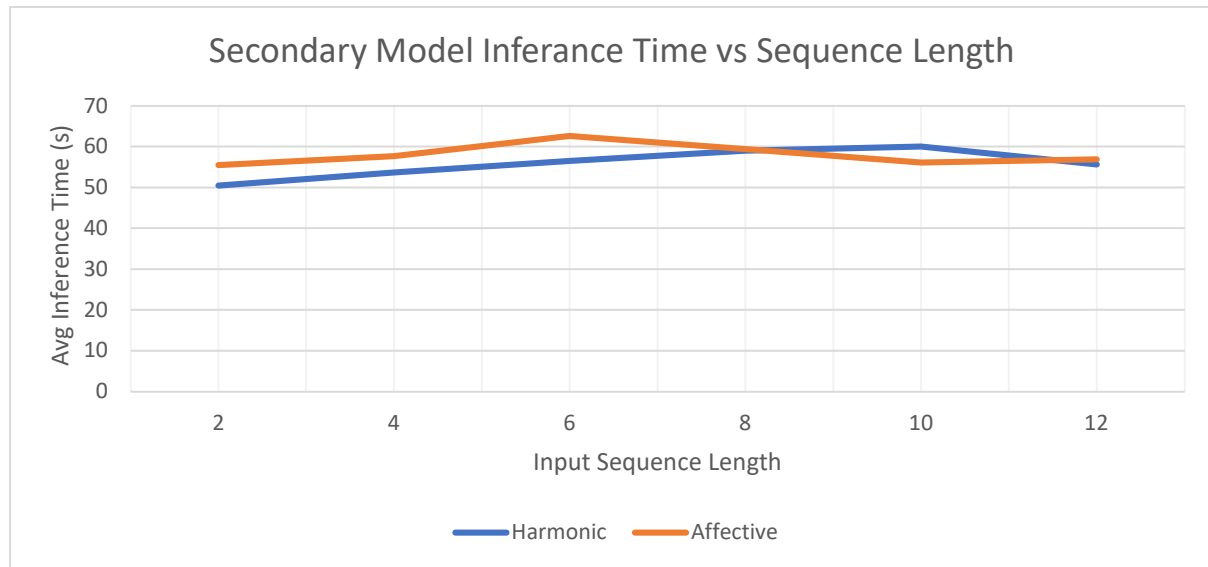


Figure 13

Secondary Model Accuracy Evaluation

While pre-loading allows the secondary model to be initialised without adding latency to user interactions, the inconsistency of the model's harmonic and affective analysis made it unsuitable for inclusion in the final implementation. **Figure 14** shows the harmonic and affective analysis produced by the secondary model on a basic four-chord sequence. **Figure 15** shows the same analysis on a more complex eight-chord sequence.

Analysing the harmonic output first. While the model correctly identified the key as C major, the numerals field returned chord root names rather than Roman numerals, where the correct output should be I, IV, V, vi. Of the four harmonic functions assigned, three were correct, however A minor was incorrectly labelled as predominant when it should be tonic, as it functions as a tonic substitute via deceptive cadence in this context. The voice leading and summary fields both simply restated the chord sequence rather than providing any analytical content. While the model demonstrates some understanding of harmonic concepts, this is a simple diatonic sequence and its failure on the A minor chord indicates it struggles with even moderately complex harmonic reasoning, making it unsuitable for implementation. These issues are more apparent in **Figure 15**. Here the JSON was invalid due to the model outputting an escaped character in the special chords field, which broke the parser. The content also deteriorated significantly: only seven functions were assigned for eight chords, and only two of those seven were correct. The relationships array collapsed to two entries instead of eight, voice

leading was truncated, and the model failed to identify any of the multiple borrowed and chromatic chords in the sequence, returning "none" for special chords. On the simple sequence the harmonic analysis achieved a 75% function accuracy rate; on the complex sequence this fell to approximately 25%.

The affective analysis performed considerably better. On the simple four-chord sequence the per-chord emotion assignments were musically justifiable, the tension arc accurately reflected the low-to-high-to-low movement of the progression, and the summary was correct if somewhat brief. The only clear failure was the overall mood field, which returned "major" — a key quality rather than an emotional descriptor, indicating the model did not follow the formatting constraint. On the complex eight-chord sequence the affective analysis still produced valid JSON and the per-chord emotions showed some sensitivity to chromatic tension, with F# minor labelled "anxious" and G minor labelled "tense." However the overall mood of "stable" is semantically incorrect for a highly chromatic progression and directly contradicts the tension arc the model itself defined. With further prompt engineering and a larger or less quantised model, the affective analysis shows more promise than the harmonic, but in its current state neither component met the standard required for integration into the software.

Overview of Secondary Model Evaluation

Overall, evaluation on the accuracy, consistency and speed of the model show that the model did not reach the requirements to be featured in the software. In order for it to be used as a trustworthy explanatory tool, it would need to reach ~99% accuracy, however tests show that the model struggles to explain even the simplest chords. Thus the explainable requirement of **A1** is not met.

LENGTH 4 | C major, F major, G major, A minor

HARMONIC (66.19s) | JSON valid: yes

Key : C major

Numerals : C, F, G, A

Functions: tonic, subdominant, dominant, predominant

Voice leading: C to F, F to G, G to A

Summary : C major to F major to G major to A minor

AFFECTIVE (62.81s) | JSON valid: yes

Mood : major

Tension arc : rising

C	happy	tension: low
---	-------	--------------

F	calm	tension: medium
---	------	-----------------

G	determined	tension: high
---	------------	---------------

A	sad	tension: low
---	-----	--------------

Summary : Emotional arc from happy to sad, with moderate emotional contrast.

Figure 14: Secondary Model's harmonic and affective analysis of a simple, four chord sequence

LENGTH 8 | C major, Eb major, A minor, F# minor, Bb major, E major, G minor, D major

HARMONIC (73.07s) | JSON valid: NO

Raw output: {"key": "C major", "numerals": ["C", "Eb", "A", "F#", "Bb", "E", "G", "D"], "functions": ["tonic", "subdominant", "predominant", "borrowed", "tonic", "subdominant", "dominant"], "relationships": ["C to Eb, Eb to A, A to F#", "C to Bb, Bb to E, E to G"], "special_chords": "none", "voice_leading": "C"}

AFFECTIVE (81.42s) | JSON valid: yes

Mood : stable

Tension arc : low

C major	hopeful	tension: low
---------	---------	--------------

Eb major	doubtful	tension: medium
----------	----------	-----------------

A minor	sad	tension: low
---------	-----	--------------

F# minor	anxious	tension: medium
----------	---------	-----------------

Bb major	determined	tension: low
----------	------------	--------------

E major	relieved	tension: low
---------	----------	--------------

G minor	tense	tension: medium
---------	-------	-----------------

D major	confident	tension: low
---------	-----------	--------------

Summary : Stable emotions throughout the progression, with a slight increase in tension during the transition from Eb major to A minor.

Figure 15: Secondary Model's harmonic and affective analysis of a complex, eight chord sequence

4d. Evaluation & User Testing of the User Interface

This section evaluates to what extent CeCe's user interface design and implementation met the goal of A3, which required the system to provide a streamlined, non-intrusive interface that supports the user's creative workflow, and NFR1, which required the interface to be easy-to-navigate. A small-scale usability study was conducted with four participants of varying musical and technical backgrounds. All participants provided informed consent prior to testing, signing the consent form seen in **Figure 20**

Methodology

Four participants were carefully selected based on their musical knowledge, two with prior musical experience and two without, in order to assess whether the interface was accessible to users regardless of musical background. Participants were recruited from the researcher's personal network and were not compensated for their time. Each participant was given a brief introduction to the purpose of the software but received no instruction on how to use the interface, so as not to influence their natural interaction with the system. They were then asked to perform a set of tasks designed to cover the core interactions with the system. These tasks included:

- Creating a new project
- Adding and renaming sections
- Navigating between section view and sequence view
- Opening the chord workshop
- Editing a chord sequence within the chord workshop
- Moving and resizing windows
- Playing, pausing, and navigating MIDI playback

Each participant's interaction with the system was observed by the researcher, and participants were encouraged to verbalise their thoughts and difficulties in real time. The completion rate of each task was recorded, along with the time taken to complete it where applicable. Following the task session, participants were asked to complete a six question questionnaire covering navigation ease, AI integration, layout clarity, moments of confusion, workflow support, and suggested improvements. The questionnaire given can be seen in **Figure 16**. All responses were anonymised prior to analysis.

Questionnaire Results

Four participants completed the usability questionnaire following interaction with the CeCe interface. Across all quantitative questions the responses were positive, with no participant rating any question below 3.

Navigation ease received a mean score of 4.25 out of 5, with 50% of participants rating it 5 and one participant each rating it 3 and 4. No participant found navigation difficult, suggesting the overall structure of the application is clear and accessible. AI integration scored the highest of any question, with 75% of participants awarding a 5 and one participant a 3, giving a mean of 4.5. This indicates that the AI suggestions were well received and did not feel disruptive to the creative process, which directly supports the creative preservation requirements of the system. Layout clarity also scored well, with responses split evenly between 4 and 5, giving a mean of 4.5 and no scores below 4. Workflow support scored a mean of 4.25, with responses of 3, 4, and two scores of 5, suggesting the interface generally supported rather than interrupted participants, though one participant found some friction in their workflow.

The qualitative responses to the confusion question revealed a consistent theme around discoverability. Three of the four participants mentioned difficulty locating specific controls, particularly the chord editor, the play button within the editor, and the editing button used to open the chord workshop. One participant noted they were unsure what the chords in the suggestion panel meant, indicating that musical terminology may not be accessible to users without a music theory background. One participant also noted they did not realise the chord workshop was a resizable window, and were unaware that chords could be inserted at any point in a sequence rather than only appended to the end.

The improvement suggestions reinforced these findings, with all four participants independently requesting clearer labelling or tooltips on buttons, and three suggesting some form of tutorial or onboarding to explain the interface and musical concepts to unfamiliar users. These responses suggest that while the core interface is well received, discoverability and onboarding are the primary areas for future development.

Observational results

The observational results, displayed in **Table 2**, presents the observed task completion times for each of the four participants across the seven defined tasks. Six of the seven tasks were completed by all four participants, with the exception of the move and resize windows task,

which P4 did not complete. This is consistent with P4's qualitative response noting they did not recognise the chord workshop as a moveable window, instead perceiving it as a fixed element of the interface. The task with the highest variance in completion time was opening the chord workshop, where times ranged from 20 seconds to 62 seconds, with P1 taking considerably longer than the other three participants. This directly corroborates the qualitative finding that locating the control used to open the chord workshop was a common point of confusion. In contrast, MIDI playback was the fastest and most consistent task, with a mean completion time of 12.25 seconds, suggesting the playback controls are intuitive and well positioned. Creating a new project and adding and renaming a section were also completed quickly and consistently across all participants, indicating that the core project management interactions present no significant usability barrier. Editing the chord sequence using AI suggestions produced the second highest mean completion time at 36.5 seconds, though times were relatively consistent across participants, suggesting this reflects the natural complexity of the task rather than interface confusion. Overall the observational data supports the questionnaire findings, with friction concentrated around window management and chord workshop discoverability rather than the core navigation and AI interaction features of the system.

Table 2

Task	P1 Completion Time	P2 Completion Time	P3 Completion Time	P4 Completion Time
Create new Project	12s	10s	22s	6s
Add & rename section	22s	18s	28s	25s
Navigate section and sequence view	20s	15s	24s	30s
Open chord workshop	62s	20s	30s	35s
Edit chord sequence using AI suggestions	40s	34s	30s	42s
Move & resize windows	25s	20s	10s	Did not complete

Play, pause & navigate MIDI playback	5s	12s	23s	9s
---	----	-----	-----	----

Limitations

Before drawing a conclusion from these results, it is important to note that this user testing has only included a small sample of possible users. There is also the problem of bias, as the participants were recruited from my personal network, and the fact of my being there when the observations were made, and the knowledge that I will be reading their feedback may have influenced participant behaviour. This will be kept in mind during my evaluation, and the conclusion drawn from this test should not be taken as objectively true.

Overall Evaluation of User Testing

The purpose of this test was to measure to what extent CeCe met the requirements of A3 and NFR1. Based on the results of both the observational study and the questionnaire, both requirements can be considered met. NFR1 required the interface to be easy to navigate, and this is supported by the navigation ease mean score of 4.25 out of 5, the consistent task completion across participants, and the absence of any score below 3 across all quantitative questions. A3 required the system to provide a streamlined, non-intrusive interface that supports the user's creative workflow. The AI integration question, which most directly measures this requirement, received the highest mean score of any question at 4.5, with 75% of participants awarding the maximum score of 5. The workflow support question further supports this conclusion with a mean of 4.25, indicating that the interface generally supported rather than interrupted the creative process.

However, the results also highlight clear areas for improvement. The primary limitation identified across both the observational data and qualitative responses is discoverability, particularly around the chord workshop entry point and window management. These are not failures of the core interface design but rather gaps in onboarding and labelling that would be addressed in future development. A secondary limitation is the accessibility of musical terminology to non-musical users, with one participant explicitly noting uncertainty around the meaning of chords in the suggestion panel.

As stated in the previous Limitations subsection, this evaluation carries inherent limitations, and a more rigorous evaluation with a larger and more diverse participant group would be required to draw stronger conclusions.

4e. Evaluation of Creative Agency Preservation

The central aim from the beginning of this project has been not simply to build an AI compositional tool, but to do so in a way which preserves the user's agency in the creative process over the AI's agency. The project aims each represent a distinct mechanism through which creative control was intended to transfer from the AI to the user. This section draws together findings from the primary model evaluation, secondary model evaluation, and the user interface evaluation to assess the extent to which this goal has been achieved.

A1 aimed to provide transparent, explainable and unique chord suggestions. The uniqueness component is well supported by the primary model evaluation, which demonstrated the full chromatic and qualitative coverage MusicLang was able to offer across all suggestions, and an estimated 2.88×10^{19} unique twelve chord sequences producible by the mode. This section showed that the mechanism through which the suggestions are generated and presented means the user is not presented with a single AI-generated answer, but with a spectrum of creative decision of varying probability, allowing them to create chord sequences they wouldn't have thought of. This directly enforces the principle of choice proliferation described in the introduction. The transparency component is partially met, while the splitting of suggestions into probability-determined categories gives the user meaningful insight into the model's confidence, it isn't able to expose the musical context behind its suggestions, as this was the job of the secondary model, which was found to be unsuitable for integration. This leads into explainability, which was not met by the system for this very reason. The partial fulfilment of transparency and the lack of fulfilment of explainability is a meaningful as without harmonic and affective explanations, users cannot learn from the AI's suggestions or develop musical understanding through the tool, which was a key part of the original vision for A1

A2 aimed to give the user fine-grained compositional control through a hierarchical song model. The abstraction of the composition into 4 hierarchical tiers, combined with the per-sequence control over tempo, time signature, and key, means the user retains meaningful authority over every structural and harmonic dimension within their composition. The ability to insert chords, sequences and sections at any position of the composition, rather than only at the end, further enforces this, ensuring the AI functions as a tool the user works with rather than a purely generative tool. FR3 and FR4 were both fully satisfied, and no usability issues

relating to compositional control were identified during user testing, supporting the conclusion that A2 was successfully met.

A3 aimed to integrate the AI non-intrusively into the user’s creative workflow through a streamlined interface. The user testing results provide the most direct evidence for this aim with the AI integration question receiving an average score of 4.5 out of 5, meaning 75% of the participants awarding the maximum mark. This indicates the AI suggestions were experienced as supportive rather than disruptive. The workflow support question scored an average of 4.25, and qualitative responses noted that the suggestions were interesting and enjoyable to explore. One participant specifically commented that being able to preview suggestions before confirming them was a positive experience, which reflects the intended design of keeping the user in control of the creative decision at every step. These results, although based on a small sample of possible users, support the conclusion that A3 was met.

Overall, the evaluation suggests that CeCe partially achieves its goal of preserving creative agency. The primary model and interface together deliver on the core promise of choice proliferation and non-intrusive assistance, and the hierarchical song model gives the user genuine compositional authority. The most significant gap is the failure of the secondary model, which leaves the explainability and the transparency requirements of A1 partly unmet. A tool that can suggest chords but not explain why they work limits the user’s ability to grow as a composer through their interactions with the system, which was central to the project’s original vision. Addressing this through a more capable model or an alternative explanation mechanism represents the most important area of future development.

5. Project Ethics

This project was conducted in accordance with the University of Liverpool’s ethical guidelines for undergraduate research. The project falls under Data Category A, as anonymous data from an open-source dataset was used during development, and Participant Category 2, as human participants were involved in the software evaluation.

The primary dataset used during the development of the primary model was Chordinomicon, a publicly available open-source chord progression dataset hosted on GitHub. No personal or sensitive data was extracted from this dataset, and the results of training on this dataset were not implemented in the final software

Human participants were involved exclusively during the user interface evaluation state. Four participants took part in an observed task session followed by a questionnaire. Prior to participation, all participants were informed of the purpose of the study, how their data would be used, and that their responses would be anonymised in any published report. Informed consent was obtained from each participant before testing began and they were told they had the ability to withdraw their consent at any time. No personally identifiable information was recorded at any stage and participants were referred to as P1 through P4 in all documentation. All response data was stored locally on the researchers device and was not shared with any third party. The small sample size of four participants was a practical constraint of the project and was taken into consideration during evaluation.

The AI models used in this project, MusicLang and Vicuna, are both open-source and freely available for academic research. Neither model was trained on proprietary data within the scope of this project, and no novel training was conducted that would raise additional ethical concern.

6. Conclusion & Future Work

6a. Summary of Achievements

This project set out to design and develop an AI-powered music composition tool that enhances the creative process while preserving the user's agency over it. The success of the project depends on implementation of three primary features - the dual-model (A1), the compositional hierarchy (A2), and the non-intrusive interface (A3)– which represent the three main aims of the project. Of the three aims, two were achieved fully. The compositional hierarchy - which split the composition into song, section, sequence, and chord, and allowed the user to modify tempo, time signature and key at the sequence level – was implemented fully and received consistently positive feedback during user testing, with participants able to complete all hierarchy-related tasks without prior instruction. This result showed on a small scale that the hierarchical representation used in the software gave the user fine-grained control over every structural and harmonic dimension of their composition, further increasing the creative agency they had over it. The non-intrusive interface also met its aim, with the streamlined window-based layout allowing AI suggestions to be presented, previewed, and accepted or rejected

without interrupting the compositional workflow. The third aim, the dual-model system, was however partially met. The primary model's temperature varying suggestion algorithm produced a diverse and musically grounded set of chord suggestions, demonstrating full chromatic coverage, no significant loss of diversity across increasing sequence lengths, and a root and quality distribution consistent with the conventions of Western tonal music. However, the secondary model was found to be unsuitable for integration, leaving the explainability dimension of A1 unmet. Overall, seven out of the ten functional requirements were fully satisfied, with one further requirement partially satisfied, reflecting a strong but incomplete realisation of the original specification.

The suggestion algorithm implemented into the primary model was the standout technical contribution of this project. Evaluation of the model's performance showed that it was able to produce a diverse and varied set of suggestions for sequences of any length and composition, demonstrating full chromatic and qualitative coverage across all test sequences. Notably, the temperature-varying approach, sampling across four temperature levels from 0.7 to 1.6, ensured that diversity was maintained even as sequence length increased, a result that would not be expected from a standard autoregressive model. The algorithm played a central role in the extent to which the software was able to address its primary goal of returning creative agency to the user. By generating up to 98 unique suggestions per request and sorting them into four probability-based categories, the algorithm directly fulfilled the choice proliferation principle described in the introduction, giving the user a wide range of genuine creative options rather than a single AI-determined answer. This is the mechanism through which CeCe most meaningfully distinguishes itself from the over-automated tools identified in the background research, where minimal user input produces a complete output with little room for creative intervention.

CeCe is able to partially address the over-automation problem identified in the introduction. Existing tools such as AIVA and MusicLM generate complete musical outputs with minimal user input, positioning the composer as a mere recipient of AI-generated material. CeCe takes a different approach and rather than generating a finished product, allows the user to select from a wide range of options at each stage of the process and leaves the final creative decision up to them. The volume and diversity of suggestions, combined with the fact that no chord is ever added without explicit user confirmation, means that the creative agency remains with the composer at every step. This shifts the roles of the AI from co-creator to assistant, which was the central goal of the project. The dual-model concept is further validated by the

user testing findings with participants explicitly noting that they would have valued explanations of what the chords meant and why they were being suggested. This goes towards confirming that the demand for explainability component of A1 exists even if the current implementation could not deliver it. This suggests that the architectural approach CeCe proposes is sound, and that that secondary model's failure was a practical limitation rather than a conceptual one. However this is just a suggestion, and for the dual-model to be truly evaluated, further development of the software is needed.

6b. Limitations

The most significant limitation of the project is the failure of the secondary model. Vicuna, even at its most optimised configuration, produced harmonic analyses that were structurally incorrect and semantically shallow, with function accuracy ranging from 25%-75% depending on the complexity of the progression. Inference times of approximately 55-58 seconds per analysis made real-time use infeasible, and even with pre-loading implemented, the quality of the outputs was insufficient to meet the standard required for integration. In hindsight, the decision to persist with a local quantised model, even when obvious flaws started appearing, was a mistake, and switching to a server-side model earlier in development would have been a better use of time and would likely have produced reliable results. Beyond the failure of the secondary model, two functional requirements, the MIDI export and archive features, were not implemented due to time constraints, and the primary model's average inference time of 6.5 seconds fell short of the 3 second target defined in NFR2. The user evaluation was also limited by its small sample size of four participants recruited from the researchers personal network, which restricts the extent to which the results can be used to justify project success. These limitations are acknowledged but do not undermine the core achievements of the project.

6c. Future Work

The most impactful direction for future development would be replacing the local secondary model with a server-side alternative. A cloud-hosted model, although possibly costly, would have sufficient capacity to perform reliable harmonic and affective analysis, fulfilling the explainability requirement of A1 and allowing the dual-model to be fully implemented and tested.. Beyond the secondary model, the two unimplemented functional requirements, MIDI export and the archive feature, although deprioritised due to their lack of connection with the primary aims, would represent relatively straightforward additions that would meaningfully

improve the completeness of the software. The user interface would also benefit from the addition of tooltips and clearer button labelling, alongside a brief introductory tutorial to familiarise the users with the tools and features of the software, these are improvements that were independently requested by all four of the participants in usability study. Finally, the primary model's inference speed could be improved through GPU acceleration or by exploring more efficient sampling strategies, bringing generation time closer to the 3 second target defined by NFR2. Together these improvements would address the most significant gaps in the current implementation and bring CeCe closer to a fully realised version of the system originally envisioned.

6d. Final Conclusion

CeCe demonstrates that it is possible to inject AI into the creative process in a way that expands rather than replaces creative decision-making. The hierarchical song model and the temperature-varying suggestion algorithm together form a foundation that genuinely shifts the role of AI from generator to assistant, placing creative authority back in the hands of the composer at every step. The secondary model's failure highlights a current limitation of small local language models for technically precise domains, but this is a limitation of the available technology, not of the dual-model concept itself. The question of whether AI can be integrated into the creative process in a way that genuinely enhances rather than impedes human creativity remains one of the most important and contested questions in the field — and one that this project was ultimately unable to fully answer. What it does demonstrate, however, is that the architectural approach proposed here is a credible and promising direction for pursuing that answer. As local model capabilities improve and server-side alternatives become more accessible, the dual-model system CeCe proposes becomes increasingly viable, and the groundwork laid by this project provides a solid foundation for the research and development still to come.

7. BCS Criteria and Critical Reflection

This section addresses each of the six BCS criteria in turn, demonstrating how the project satisfies them, and concludes with a critical self-evaluation of the development process.

An Ability To Apply Practical and Analytical Skills Gained During The Degree Program.

The development of CeCe required the application of skills across multiple areas covered in the degree. The most prominent of which was from the Software Engineering 1 and 2 modules, from which many software engineering principles such as Open/Closed, SOLID and Agile, were used to ensure that the software was extendable and coherent. The design and implementation of the primary and secondary models required analytical understanding of transformer model outputs, probability distributions, and temperature-scaled sampling, all of which was learnt from various areas of the course. The evaluation methodology, including the design of modularised tests, analysis of model outputs, and the structured usability study, further demonstrates the application of analytical skills developed throughout the program

Innovation and/or Creativity

The primary innovation of this project lies in the temperature-varying suggestion algorithm built on top of the MusicLang model. Rather than simply integrating a pre-trained model and exposing its top prediction, a custom algorithm was designed that extracts a diverse range of suggestions across four temperature levels, deduplicates, and categorises them by probability into human-readable format. The dual-model architectural concept also represents a novel approach to a problem of over-automation in AI music tools that is not widely implemented in existing software.

Synthesis of Information, Ideas and Practices to Provide A Quality Solution Together With an Evaluation of that Solution

CeCe synthesises ideas from music theory, machine learning, software architecture, and user interface design into a single cohesive system. The design decisions made throughout development, including the choice of layered architecture, the hierarchical song model and the prompt engineering applied to the secondary model, were each informed by research into existing literature and tools. The evaluation chapter then systematically assessed the resulting solution against both functional and non-functional requirements, using a combination of automated testing scripts, model output analysis, and a structured usability study to produce a thorough and evidence based verdict on the systems performance.

The Project Meets Needs in Wider Context

The need this project addresses is identified and evidenced in the introduction through a review of existing literature. Studies consistently note that current AI music tools over-automate the creative process and reduce user agency, and this gap is further supported by the lack of existing tools that implement generative AI alongside explanations in a non-intrusive way. CeCe directly targets this gap, and the user testing results, particularly the qualitative responses indicating that participants would have valued chord explanations, further confirm that the need for this kind of tool exists among real users.

An Ability to self-manage a significant piece of work

CeCe represents a significant and complex piece of individual work, spanning frontend development in Electron and React, backend AI integration in Python, machine learning model evaluation, and academic writing. Development was tracked throughout using GitHub as version control, and an iterative approach was adopted that allowed design decisions to inform one another in a sequence. In areas where the project was well-managed, this was most evident in the frontend architecture, the decision to implement and validate the layered architecture and window system early meant that subsequent features could be built on stable foundation without revisiting earlier decisions. However, self-management was notably weaker during the secondary model phase of development. Rather than setting a clear performance threshold and switching approach once it became apparent that vicuna would not meet it, too much time was spent attempting to optimise a model that was fundamentally unsuited to the task. A more

disciplined project management approach, such as defining explicit criteria for each component before development began, would have prevented this and likely resulted in a more complete final implementation. The experience highlighted that strong technical execution in one area of the project does not automatically translate to effective management of the project as a whole

Critical Self Evaluation of the Process

Reflecting on the project as a whole, I can identify areas of genuine strength, and areas where a different approach and mindset would have produced better, more complete outcomes. The design and implementation of the suggestion algorithm and the hierarchical song model are both contributions I am most proud of. Both were implemented methodically, evaluated rigorously, and performed in line with or beyond my expectations. The architectural decisions I made early in development, specifically my decoupled 4 layer structure and use of finite state machines for window management, reflected sound software engineering judgement and directly contributed to the extensibility and stability of the final system.

However, my handling of the secondary model represents the clearest failure of the project. The decision to use a local quantised LLM for a task requiring precise, structured music theory analysis was overly optimistic, and more importantly it was based on the fact that I wanted the software to function offline and be free to use. My failure to recognise early enough that the model was fundamentally unsuitable and my failure to change mindset and switch to a server side model significantly cost development time that could have been better spent elsewhere. In hindsight, I should have conducted a brief feasibility study at the outset, testing whether a small local model could reliably produce structured JSON music theory analysis before committing to it as the core implementation strategy. This would have identified the problems far earlier and allowed me to pivot to a server side alternative while there was still time to implement it properly.

More broadly, this project revealed a tendency in myself to invest effort in solving individual technical problems rather than stepping back to question whether the problem needed to be approached differently. Recognising this is something I intend to carry forward into future projects, and I plan to set explicit performance goals for each component in early development, and treat failure to meet those thresholds as a signal to reassess the approach rather than redouble optimisation efforts. This is something that will benefit both me and my future projects, and I am grateful this project has given me a chance to reflect on it.

References

- [1] Anantrasirichai, N. and Bull, D. (2021). Artificial Intelligence in the Creative industries: a Review. *Artificial Intelligence Review*, [online] 55(1), pp.589–656. doi:<https://doi.org/10.1007/s10462-021-10039-7>.
- [2] Huang, M.H. and Rust, R.T. (2018). Artificial Intelligence in Service. *Journal of Service Research*, [online] 21(2), pp.155–172. doi:<https://doi.org/10.1177/1094670517752459>.
- [3] McCormack, J., Gifford, T. and Hutchings, P. (2019) Autonomy, Authenticity, Authorship and Intention in Computer Generated Art. In: Ekárt, A., Liapis, A. and Castro Pena, M.L., Eds., *Lecture Notes in Computer Science*, Springer, 35–50. https://doi.org/10.1007/978-3-030-16667-0_3
- [4] Agostinelli, A., Denk, T.I., Borsos, Z., Engel, J., Verzetti, M., Caillon, A., Huang, Q., Jansen, A., Roberts, A., Tagliasacchi, M. and Sharifi, M., 2023. Musiclm: Generating music from text. *arXiv preprint arXiv:2301.11325*.
- [5] Dhariwal, P., Jun, H., Payne, C., Kim, J.W., Radford, A. and Sutskever, I., 2020. Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*.
- [6] Herremans, D., Chuan, C.H. and Chew, E., 2017. A functional taxonomy of music generation systems. *ACM Computing Surveys (CSUR)*, 50(5), pp.1–30.
- [7] QIU, J. (2023). *Modularizing React Applications with Established UI Patterns*. [online] martinoflower.com. Available at: <https://martinoflower.com/articles/modularizing-react-apps.html> [Accessed 24 Nov. 2025].
- [8] Ens, J. and Pasquier, P. (2020) *The Chordinator: Chord progression modelling and generation using transformers. Proceedings of the 1st Workshop on AI Music Creativity*. Available at: <https://arxiv.org/abs/2007.14822>
- [9] *Chordinomicon (2021) Chord progression dataset*. GitHub repository. Available at: <https://github.com/chordinomicon/chordinomicon>
- [10] MusicLang (2025). *GitHub - MusicLang/musiclang*. [online] GitHub. Available at: <https://github.com/MusicLang/musiclang> [Accessed 5 Dec. 2025].
- [11] Abramov, D. and Clark, A., 2015. *Redux: A predictable state container for JavaScript apps*. [online] Available at: <https://redux.js.org/> [Accessed 14 May 2026].
- [12] Weststrate, M., 2015. *MobX: Simple, scalable state management*. [online] Available at: <https://mobx.js.org/> [Accessed 14 May 2026].
- [13] Davis, N., 2013. Human-computer co-creativity: Blending human and computational creativity. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 9(6), pp.9–12. <https://doi.org/10.1609/ai-ide.v9i6.12603>
- [14] Kantosalo, A. and Toivonen, H., 2016. Modes for creative human-computer collaboration: Alternating and task-divided co-creativity. In: F. Pachet, A. Cardoso, V. Corruble and F. Ghedini, eds., *Proceedings of the Seventh International Conference on Computational Creativity (ICCC 2016)*. Paris: Sony CSL, pp.77–84. Available at: <http://www.computationalcreativity.net/iccc2016/wp-content/uploads/2016/01/Modes-for-Creative-Human-Computer-Collaboration.pdf>
- [15] Shneiderman, B., 2007. Creativity support tools: Accelerating discovery and innovation. *Communications of the ACM*, 50(12), pp.20–32. <https://doi.org/10.1145/1323688.1323689>
- [16] Roberts, A., Engel, J., Mann, Y., Gillick, J., Kayacik, C., Nørly, S., Dinculescu, M., Radebaugh, C., Hawthorne, C. and Eck, D., 2019. Magenta Studio: Augmenting creativity with deep learning in Ableton Live. *Proceedings of the 6th International Workshop on Musical Metacreation (MUME 2019)*, Charlotte, USA, 17 June 2019. <https://doi.org/10.5281/zenodo.4285266>
- [17] Gleissner, B., 2017. *Soundfont-Player: A JavaScript soundfont player for browsers*. GitHub repository. Available at: <https://github.com/gleitz/midi-js-soundfonts> [Accessed 14 May 2026].
- [18] Adobe (2025). *Color Contrast Analyser*. [online] Adobe Color. Available at: <https://color.adobe.com/create/color-contrast-analyzer> [Accessed 14 May 2026].

Appendix

```
//binds the actions into a single dictionary
const boundActions = {}

for (const [name, action] of Object.entries(actions)) {
  boundActions[name] = (...args) => {
    const fn = action(...args);

    //actions mutate the state directly
    const result = fn(stateHolder.state, domain, boundActions, setState);

    //calls publisher
    notify();
    stateHolder.state.app.version++;
    //return result if action returns a value
    return result;
  };
}
```

Figure 2: Creation of Bound Actions within the Global Store

```
export const testAction = (params) => (state, domain, actions) => {
  //action code
}
```

Figure 3: Test Action Extracting Global Store dependencies

```

// Handles window state
export function transition(state, id, event) {
  const win = {...state.windows.byID[id]};
  switch (win.state) {
    case "normal":
      if (event === "START_MOVE") win.state = "moving";
      if (event === "START_RESIZE") win.state = "resizing";
      if (event === "MAXIMISE") win.state = "maximised";
      if (event === "DOCK") win.state = "docked";
      if (event === "POP_OUT") win.state = "popped"
      break;

    case "moving":
      if (event === "STOP") win.state = "normal";
      if (event === "DOCK") win.state = "docked";
      break;

    case "resizing":
      if (event === "STOP") win.state = "normal";
      break;

    case "docked":
      if (event === "START_UNDOCK_DRAG") win.state =
"undocking";
      break;

    case "undocking":
      if(event === "STOP") win.state = "docked";
      if (event === "UNDOCKED") win.state = "moving";
      break;

    case "maximised":
      if (event === "RESTORE") win.state = "normal";
      break;

    case "popped":
      if (event === "POP_IN") win.state = "normal";
  }
  return win;
}

```

Figure 4: Transition Function for handling window state within the Domain layer

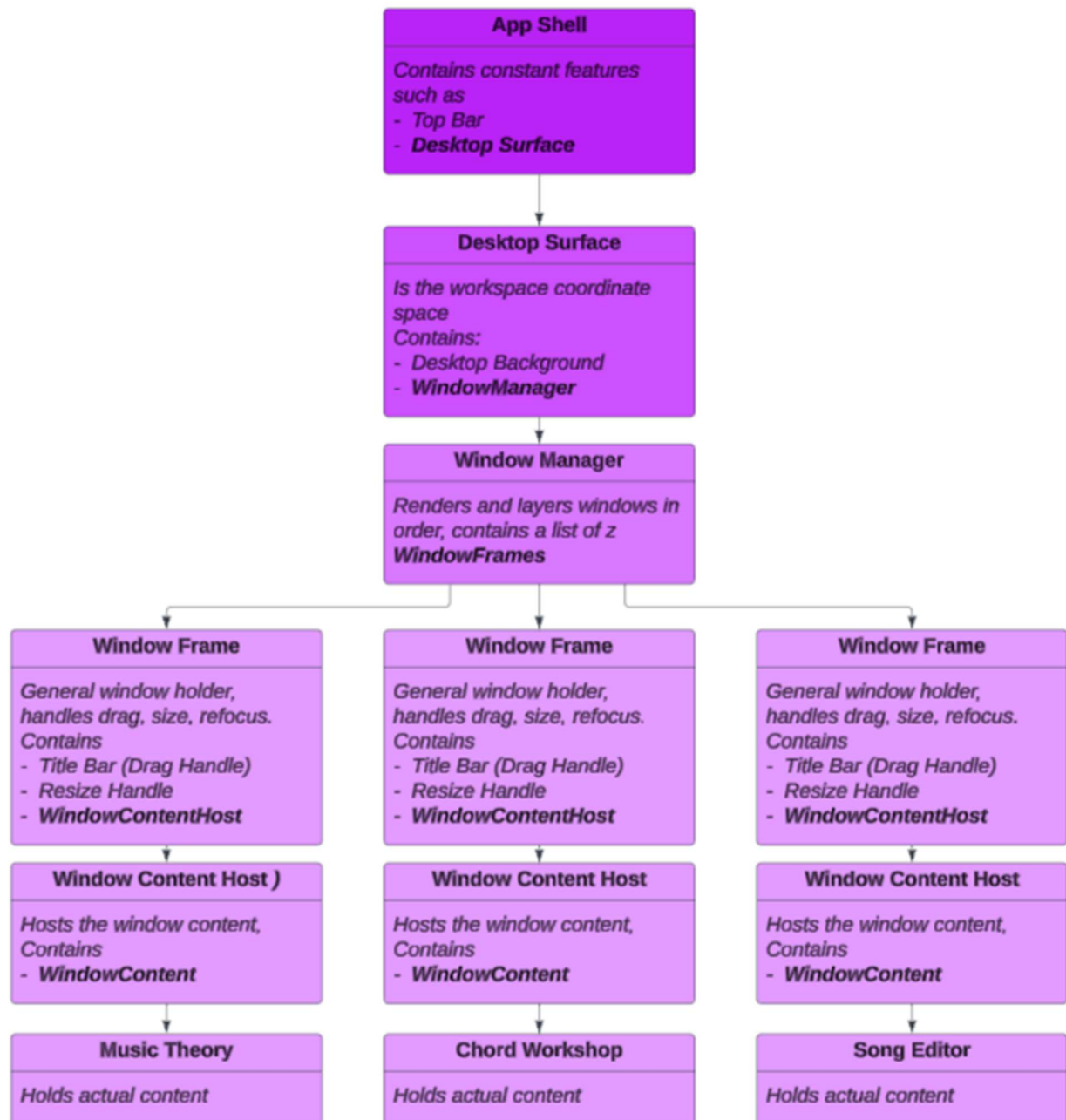


Figure 5: View Layer Architecture

```

def predict_chord(chordDSL: str):
    ## Tokenises the input sequence using the custom tokenizer
    sequence_base = chord.chords_to_tokens(chordDSL)
    new_chord = []
    temp = 1
    output = []
    ## Four temperature levels for diverse suggestion generation
    temperatures = [0.7, 1.0, 1.3, 1.6]
    ## Weight adjustment for different temperatures
    weights = [1/t for t in temperatures]

    for x in range(4):
        ## Sets the correct temperature and weight for the current batch
        temp = temperatures[x]
        weight = weights[x]
        for i in range(40):
            ## Creates fresh copy of input sequence
            sequence = sequence_base.copy()
            new_chord = []
            log_prob = 0
            ## Extracts all 6 tokens to form a full chord
            for i in range(6):
                logits = get_logits(" ".join(sequence))
                next_position = len(sequence) % 6

                ## Applies position mask to constrain valid tokens at each position
                masked_logits = decode.apply_position_mask(logits, next_position, VOCAB)

                ## Scales logits by temperature before sampling
                masked_logits = decode.apply_temperature(masked_logits, temp)

                ## Converts logits to probability distribution
                probs = torch.softmax(masked_logits, dim=-1)
                ## Samples next token from distribution
                next_id = torch.multinomial(probs, num_samples=1).item()
                ## Accumulates log probability of the chord
                log_prob += torch.log(probs[next_id]).item()
                next_token = ID_TO_TOKEN[next_id]
                new_chord.append(next_token)
                sequence.append(next_token)

            ## Applies weight and decodes tokens back to chord representation
            output.append([log_prob * weight, decode.decode_token_result(new_chord)])
        ## Deduplicates and sorts by probability
        output = decode.get_unique_probabilities(output)

    return output

```

Figure 8: Temperature-Varying Algorithm used to extract chord suggestions from MusicLang

CeCe User Interface and Experience Feedback

This form contains 5 questions about the software you have just interacted with. Results will be used to evaluate the software as part of a university dissertation. All responses will be anonymized in the report.

louisa@wood@gmail.com [Switch account](#)

Not shared

* Indicates required question

How easy was it to navigate between the different sections of the application? *

(1-5, where 1 = very difficult, 5 = very easy)

1

2

3

4

5

☐
☐
☐
☐
☐

How well did the AI suggestions integrate with your creative process? *

(1-5, where 1 = not integrated at all, 5 = very well integrated)

1

2

3

4

5

☐
☐
☐
☐
☐

How clear was the layout and visual organisation of the interface? *

(1-5, where 1 = very unclear, 5 = very clear)

1

2

3

4

5

☐
☐
☐
☐
☐

Were there any moments where you were unsure what to do next or where to find something? *

Your answer

Overall, how well did the interface support rather than interrupt your workflow? *

(1-5, where 1 = very interrupting, 5 = very supportive)

1

2

3

4

5

☐
☐
☐
☐
☐

Is there anything you would change or improve about the interface?

Your answer

Submit

Clear form

Figure 16: Questionnaire given to the user study participants

```
// Sort by likelihood DESC (higher = more likely)
entries.sort((a, b) => b[1] - a[1]);

const total = entries.length;

const q1 = Math.floor(total * 0.25);
const q2 = Math.floor(total * 0.50);
const q3 = Math.floor(total * 0.75);

state.workshop.recommendedChordData = {
  Normal: entries.slice(0, q1).map(([chord]) => chord),
  Uncommon: entries.slice(q1, q2).map(([chord]) => chord),
  Strange: entries.slice(q2, q3).map(([chord]) => chord),
  Unadvisable: entries.slice(q3).map(([chord]) => chord),
};
```

Figure 17: Code showing how suggestions are split into probability defined groups



School of Computer Science & Informatics
Ashton Street, Liverpool, L69 3DR

Computer Science Project Modules Participant Consent Form

Project Title: Building an AI-Powered Song Writing Tool: Enhancing Creativity Without Impeding Humanity

Student: Louis J Selwood

Supervisor: Blaine Keetch

Coordinator: Dr Stuart Thomason (s.thomason@liverpool.ac.uk)

Instructions to student: Mark the boxes that apply to your project so the participant knows what you will be asking them to do. If you later decide to ask the participant to do something that isn't marked, fill in a new form.

You are inviting the participant to take part in...

(Copy/paste the ☒ symbol where necessary)

☐ Requirements Analysis ☒ Software Evaluation

You plan to use the following methods as part of this activity...

☐ Interview ☐ Discussion ☒ Observation
☐ Narration ☒ Questionnaire

You will be using the following techniques to record what the participant is saying or doing during the activity...

☐ Audio Recording ☐ Screen Recording ☒ Written Notes
☐ Apple TestFlight or Google Console (for which the participant's Apple or Google ID will be required)

Instructions to participant: Please read the Participant Information Sheet carefully. If you agree to take part in the activities indicated above, enter your initials in the box below and return this sheet to the student.

- 1) I confirm that I have read and understood the information sheet attached to this form, or it has been read to me. I have had the opportunity to consider the information and ask questions. My questions have been answered to my satisfaction.
- 2) I understand that my participation is voluntary and that I am free to stop taking part at any time without giving a reason. In addition, I understand that I do not have to answer all questions or take part in all listed activities.
- 3) I understand I can ask to see the information I have provided, and I can ask for it to be destroyed before it is aggregated and anonymised. I understand that after the information has been anonymised, I can no longer ask for it to be destroyed.
- 4) I understand that the information I provide will be stored in line with the data protection policy of the University of Liverpool until it is destroyed.
- 5) I understand that my personal contribution will be destroyed when it has been aggregated and anonymised.
- 6) I agree that this consent form will be retained by the project supervisor for a period of two months following submission of the dissertation of which it forms part, after which it will be destroyed.

☒ I agree to take part in these activities
(enter your initials into the box)

Figure 20: Screenshot of Participant Consent Form Signed